

Joseph F. Rudzinski Nicholas J. H. Dunn Michael R. DeLyser
Ryan J. Szukalo Maria C. Lesniewski Will G. Noid

[illegible]

Contents

1	Installation	5
2	Getting Started	6
2.1	Main Flow Chart	6
2.2	File Type Overview	7
2.2.1	Trajectory File (.trr)	7
2.2.2	BOCS Trajectory File (.btj)	7
2.2.3	GMX Topology File (.top)	7
2.2.4	BOCS Mapping File (map.top)	7
2.2.5	BOCS Topology File (.btp)	7
2.2.6	Force-Matching Input Files (par.txt, inp.txt)	8
2.3	Executable Overview	8
2.3.1	translator	8
2.3.2	cgmap	8
2.3.3	cgff	8
2.3.4	tables	8
2.3.5	ldcalc	9
2.3.6	LDPDF	9
2.4	Pressure-Matching	9
2.4.1	pmatch	9
2.4.2	Input File (config.ini)	9
2.4.3	Output File (psi.dat)	9
3	Translator	11
3.1	Basic Usage	11
3.1.1	Case 1: Translating a topology file	11
3.1.2	Case 2: Translating a trajectory file	12
3.1.3	Units	12
3.2	Topology Requirements	12
3.3	BOCS Topology File (.btp) Format	13
3.3.1	[.btp directives]	16
4	CGMap	20
4.1	Basic Usage	20
4.2	Mapping Topology File Format	20
4.2.1	Multiple molecules in a system	22
5	CGFF	24
5.1	Input Files	24
5.2	List of Directives	24
5.3	Mandatory Directives	26

5.4	Dependent Directives	30
5.5	Optional Directives	32
5.6	Example par.txt and inp.txt files	42
5.7	Output Files	44
5.7.1	summarize_input.txt	47
5.7.2	log.txt	49
5.7.3	out.total.txt	50
6	Tables	51
6.1	Non-bonded Interactions	51
6.2	Bonded Interactions	51
6.3	Usage	52
6.4	Examples	52
6.5	Converting to LAMMPS tables	53
6.5.1	Pair/Intramolecular Interactions	53
6.5.2	Local Density Potentials, Square Gradient Interactions	53
7	Pressure-Matching	54
7.1	Introduction	54
7.2	Input file: *.ini	54
7.2.1	Universal settings	55
7.3	Example config.ini file	56
7.4	Output file: psi	57
7.5	Output file: Q	58
8	Modified LAMMPS	58
8.1	Pressure Matched Models	59
8.1.1	Analytic F_V	59
8.1.2	Tabulated F_V	59
8.1.3	Thermo_modify	60
8.2	Local Density (LD) potentials	61
8.2.1	Processing LD trajectory info in/with LAMMPS	61
9	Tutorials	64
9.1	Lennard-Jones Recovery Tutorial	64
9.1.1	AA Simulation	64
9.1.2	Force Matching	64
9.2	1-Site Methanol Tutorial	66
9.2.1	AA Simulation	66
9.2.2	Mapping	66
9.2.3	Force matching	67
9.2.4	Test Simulation	68
9.3	Extended Ensemble and PMatching Tutorial: Heptol Mixtures	69

9.3.1	AA Simulations	70
9.3.2	Mapping	70
9.3.3	Force Matching	71
9.3.4	Make Table Files	72
9.3.5	Pressure Matching: Rerun	73
9.3.6	Pressure Matching: Zeroth Iteration	75
9.3.7	Pressure Matching: First Iteration	76
9.4	Local Density Tutorial	77
9.4.1	AA Simulation / Mapping	78
9.4.2	Calculating Local Densities	78
9.4.3	Force Matching	81
9.4.4	Preparing CG Interaction Tables for LAMMPS simulations	82
9.4.5	Running CG NPT LAMMPS Simulations with LD Potentials	83
9.5	External-Fields Recovery Tutorial	84
9.5.1	Overview	84
9.5.2	External-Fields Recovery Procedure	85

1 Installation

The BOCS toolkit employs CMake (<https://cmake.org>) for its build system. To compile the BOCS toolkit, it requires a C/C++ compiler (gcc), openmpi, and a linear algebra library (MKL or LAPACK). After pulling the latest code from the Github repository, there are two separate folders:

- `force-matching`
- `pressure-matching`.

To compile, the following protocol can be followed for both toolsets. Navigate into the directory of interest (`force-matching` or `pressure-matching`). Modify the `CMakeLists.txt` control file to reflect the architecture of the system on which BOCS is being compiled such that:

1. Linear Algebra Library

Case 1: MKL (default)

Modify line 33 to point to the root directory of the MKL library. This directory should contain the file `libmkl_rt.so`

Case 2: LAPACK

Comment out line 21.

Modify line 51 to point to the root directory of the LAPACK library.

2. TIRPC Library

Modify line 24 to point to the root directory of the parent of `/include/tirpc` directory. From the location, you should be able to `ls /include/tirpc/rpc/xdr.h` and `ls /lib/libtirpc.so` or `ls /lib64/libtirpc.so`

Once the `CMakeLists.txt` file has been modified, run the commands:

```
mkdir build
cd build
cmake .. -G "Unix Makefiles" -DCMAKE_C_COMPILER=mpicc -DCMAKE_INSTALL_PREFIX=build
make
```

If all goes well, the code should then compile without errors. The pressure-matching code is bundled separately from the other coarse-graining tools, but is built in the same way.

Please note that this software has only been tested on Linux systems.

2 Getting Started

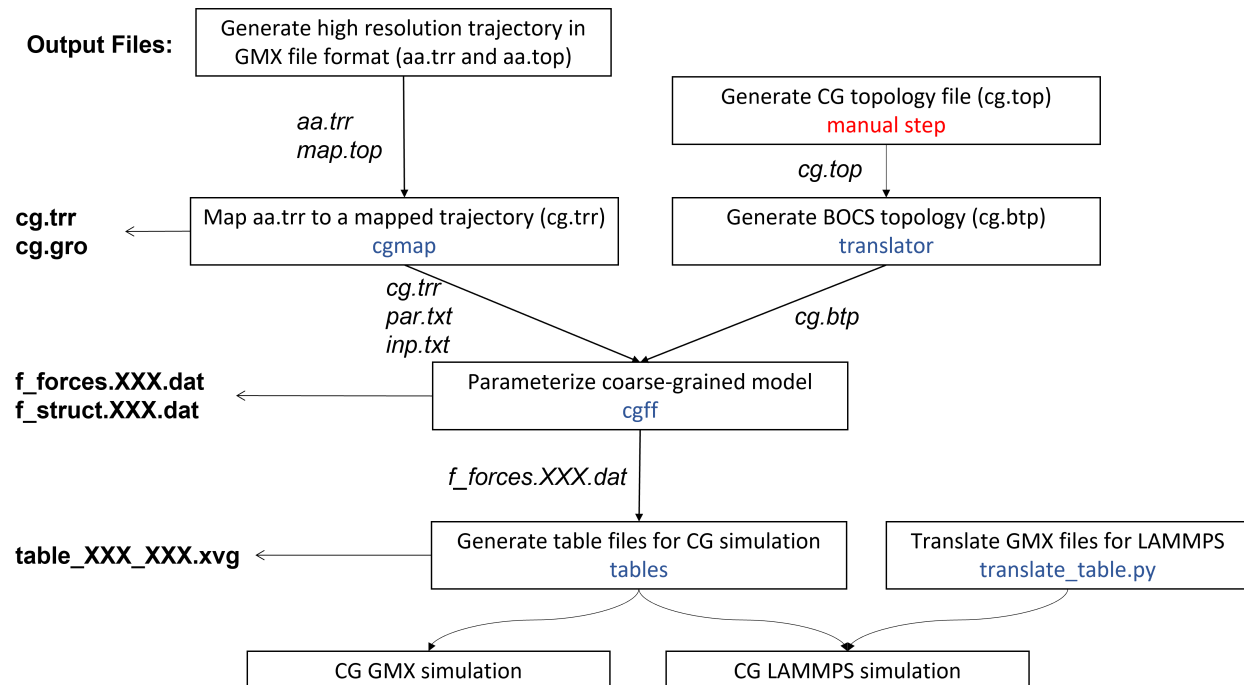


Figure 1: Flow chart of the typical BOCS toolkit work-flow for parameterizing a bottom-up CG model. Blue text corresponds to BOCS executables. Italicized words correspond to input files while the bolded words correspond to output files.

The Bottom-Up Open-Source Coarse-Graining (BOCS) software is an open-source toolkit for parameterizing bottom-up coarse-grained (CG) forcefields by providing robust implementations of the multi-scale coarse-graining (MS-CG) method and the generalized-Yvon-Born-Green (g-YBG) method. Additionally, the BOCS toolkit provides:

1. An extended ensemble capability which allows for model parameterization from multiple input models.
2. Pressure-Matching (PM): which allows for an accurate calculation of pressure-volume equation of states for homogeneous systems.
3. Energy-Matching (EM): which provides an implementation for an energy-matching variational principle to directly estimate the energetic contribution to the CG forcefield.

2.1 Main Flow Chart

Figure 1 presents the general work-flow when using the BOCS toolkit to parameterize a bottom-up CG model from a single reference high resolution model. BOCS has re-purposed

many of the Gromacs data structures and therefore uses some Gromacs file types. BOCS also provides tools to convert simulation files from Gromacs to LAMMPS format to enable CG simulations with the LAMMPS simulation engine, however in its current state, BOCS requires the input trajectories to be in the Gromacs .trr file type.

2.2 File Type Overview

BOCS leverages many existing file types from the Gromacs molecular dynamics software. Here is an overview of the important files employed in the work-flow.

2.2.1 Trajectory File (.trr)

The standard output trajectory file format from Gromacs that is used in the BOCS work-flow. This is a binary trajectory file (.trr) that can be dumped to a text file using Gromacs tools. This file contains the positions, velocities and sometimes the forces for each particle saved from a simulation.

2.2.2 BOCS Trajectory File (.btj)

This is a text trajectory file that contains system information and relevant trajectory information for each frame of a trajectory. This file can contain local densities and/or gradients of the local densities and is to be used as input to force-matching if local density or gradient potentials are to be parameterized. The BOCS translator can generate this file from a .trr file OR it can be generated by the `ldcalc` executable.

2.2.3 GMX Topology File (.top)

The molecular topology file contains a complete description of all the interactions in your system. Gromacs requires an atomistic topology file for any atomistic simulation, while BOCS can use a CG topology (.top) file to create a BOCS topology (.btj) file, which is required for the force-matching protocol (see Figure 1).

2.2.4 BOCS Mapping File (map.top)

This file contains the user-defined mapping for each unique type of molecule in the system. This file is to be created manually. The basic layout of the mapping topology file is similar to a simplified Gromacs topology (.top) file. The `cgmap` program uses the mapping topology file to generate the mapped ensemble to be used as input into force-matching.

2.2.5 BOCS Topology File (.btp)

This file is analogous to a GMX topology file but is the BOCS topology file format. It contains much of the same information as a .top file and is required for the `cgff` force-

matching tool. It is most easily generated from a Gromacs dumped .tpr file, which is a converted text file summarizing a Gromacs simulation.

2.2.6 Force-Matching Input Files (`par.txt`, `inp.txt`)

The two files `par.txt` and `inp.txt` are mandatory input files for the `cgff` tool and must be named as such. The `par.txt` file contains a list of directives and parameters that are to be employed during the force-matching procedure. The `inp.txt` file contains only one directive and simply lists the trajectory files (or path to) to be force-matched.

2.3 Executable Overview

BOCS currently contains eight different executables. The six shown in Fig. 1 comprise the main part of the toolkit and are employed in the general work-flow to parameterize a CG model from a given atomistic model. These four are introduced in this section, while the rest will be introduced in Sections 2.4 when discussing the pressure-matching functionality of BOCS.

2.3.1 translator

The `translator` tool is used to convert topology and/or trajectory files between Gromacs, LAMMPS, and BOCS file formats.

2.3.2 cgmap

The `cgmap` tool maps an all-atom trajectory file to its coarse-grained representation according to the given `map.top` file. It takes as input an all-atom trajectory and a mapping topology and generates a CG trajectory file and/or a single frame CG.gro file containing the coordinates of the mapped CG sites for the last frame of the input trajectory. The `cgmap` tool can also write a binary file containing the local density and/or gradient of the local density if either of the two types of potentials are being included.

2.3.3 cgff

The `cgff` tool derives a force field from any input trajectory using force matching and/or the g-YBG method.

2.3.4 tables

The `tables` tool creates Gromacs table files for non-bonded and bonded interactions from the tabulated force files that are output from the `cgff` program. Currently, this program only produces Gromacs non-bonded and bonded pair interaction table files and cannot handle local density or square gradient forces as input. This functionality will hopefully be added in eventually.

2.3.5 ldcalc

This `ldcalc` tool calculates the local density for each particle in an input simulation and outputs a BOCS trajectory (.btj) file. It also outputs the high and low values of the local density to use as an input into a force-matching calculation if parameterizing with local density forces.

2.3.6 LDPDF

The LDPDF tool is a simple analysis tool that takes in a BOCS trajectory (.btj) or dumped lammps file that contains local densities and computes a normalized local density probability distribution function. It can likewise do this for the magnitude of the square gradients stored in the trajectory.

2.4 Pressure-Matching

This section introduces the pressure-matching tool in the BOCS toolkit. Figure 2 outlines the general pressure-matching work-flow used to determine a configuration-independent volume potential, $U_V(V)$, that allows CG models to sample isotropic volume fluctuations under constant external pressure. The algorithm requires a fixed effective CG potential which can be determined using BOCS (1) or some other coarse-graining method. The pressure-matching algorithm is iterative and requires user analysis to determine convergence of the pressure correction. Finally, the resulting CG model can be simulated with any LAMMPS distribution that has included the BOCS package when compiling, see LAMMPS documentation for more details.

2.4.1 pmatch

The `pmatch` tool must be compiled separately to the rest of the BOCS toolkit and determines a configuration-independent volume potential that allows CG models to accurately model pressure-volume equation of states for homogeneous systems. Models which include this volume potential must be simulated in LAMMPS with the BOCS package included from the official LAMMPS distribution.

2.4.2 Input File (config.ini)

The required input text file to run the pressure matching tool. This file is passed as an argument into the pressure-matching algorithm and contains all parameters for the calculation.

2.4.3 Output File (psi.dat)

The text file with the results of the pressure-matching tool. The file contains the two psi coefficients which determine the quadratic volume potential as:

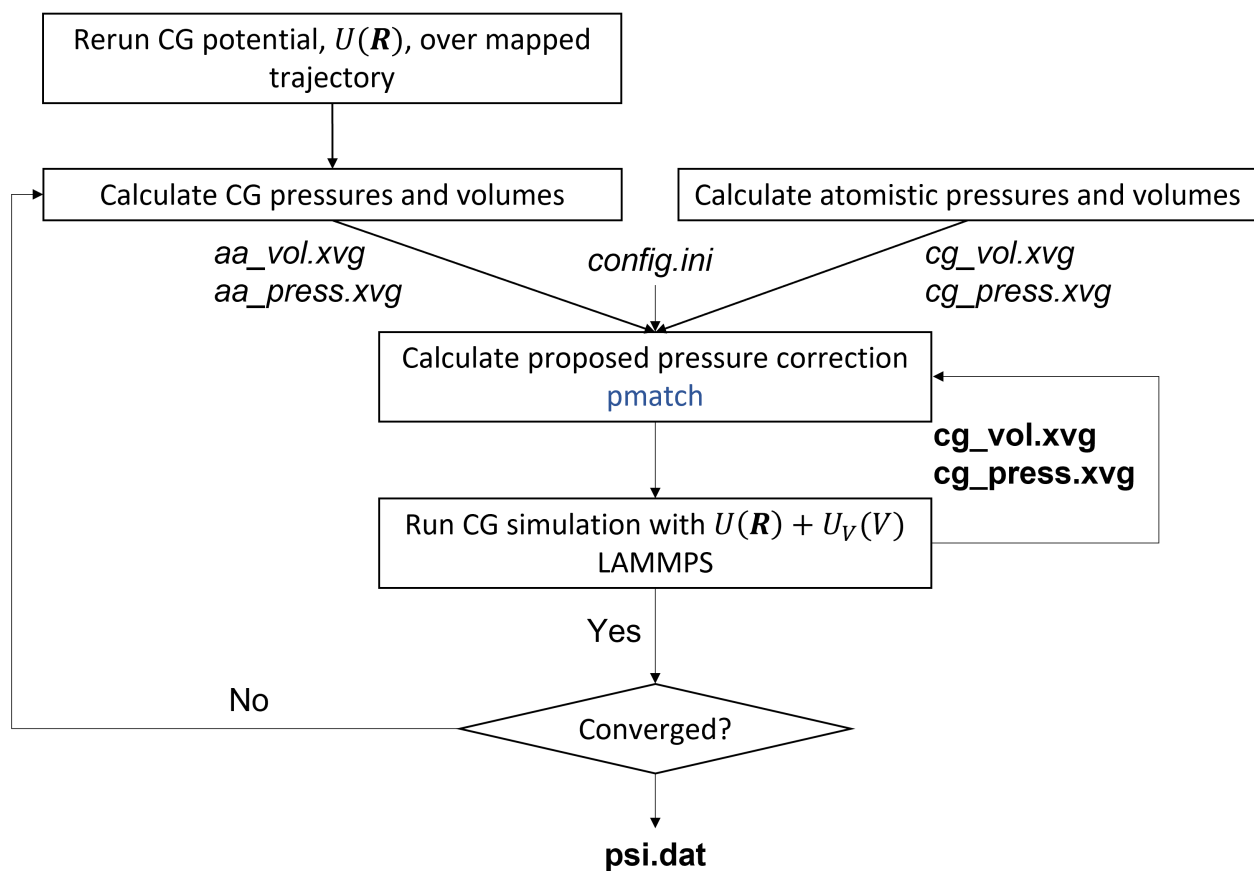


Figure 2: Flow chart of the pressure-matching work-flow for parameterizing a configuration-independent volume potential. Blue text corresponds to BOCS executables. Italicized words correspond to input files while the bolded words correspond to output files.

$$U_V(V) = \sum_d \psi_d \omega_d(V) \quad (1)$$

such that

$$\omega_d(V) = \begin{cases} N(V/\bar{v}) & \text{for } d = 1 \\ N(V/\bar{v} - 1)^d & \text{for } d \geq 2 \end{cases} \quad (2)$$

where V is the CG volume and $\bar{v}$ is the average volume of the atomic model.

3 Translator

The `translator` tool is used to convert either topology or trajectory files from one format to another. See Table 1 for all possible flags and their uses. Currently, the translator tool supports a variety of binary and text file types from Gromacs and LAMMPS, as well as BOCS trajectory and topology files, `.btj` and `.btp`, respectively.

Table 1: The possible command line flags available for use with the translator program, and what each specifies.

Flag	Meaning	Type	Description
-f	input filename	string	The name of the file you want to translate.
-o	output filename	string	The name of the resulting topology, trajectory, or structure file.
-s	topology	string	The name of BOCS topology file (<code>.btp</code>) corresponding to the trajectory.
-h	help		Print this table and quit.

3.1 Basic Usage

3.1.1 Case 1: Translating a topology file

Figure 1 shows that the force-matching code requires a BOCS topology file (`.btp`). The main use of the translator tool is to convert a Gromacs `.tpr` file to a `.btp` file. Currently, the only topology format that can be read by (input to) the translator is the Gromacs “dump” format. The “dump” format is how the information is presented whenever you run the GROMACS program `dump` with a `tpr` file:

```
gmxdump -s cg.tpr > cgdump_tpr.txt
```

The `translator` tool can then be used to convert the dumped `.tpr` file to the BOCS (`.btp`) format. The `translator` tool infers the file types to be converted by the file extensions provided, i.e. the only way the program knows that you are trying to create a `.btp` file is if your output file name (specified with the “-o” flag) ends in “`.btp`”. It will then assume the input file is a dumped `.tpr` file.

```
translator -f cgdump_tpr.txt -o cg.btp
```

3.1.2 Case 2: Translating a trajectory file

The translator tool can also convert between various Gromacs, LAMMPS, and BOCS structure and trajectory files. Table 2 lists all file formats currently supported by the translator tool. Note, a BOCS topology file (.btp) is required if your output file is a “.gro”, “.lmp”, or “.data” file. The basic syntax to is:

```
translator -f cg.trr -o cg.btj -s cg.btp
```

Table 2: The file type extensions.

Extension	Meaning	Description
.btp	BOCS topology	A text file containing the system topology information.
.btj	BOCS trajectory	A text file that contains relevant trajectory information.
.gro	Gromacs’ configuration file format	This is the standard .gro file format.
.trr	Gromacs’ standard binary format	The typical binary trajectory format used by Gromacs.
.dump	Gromacs’ dumped trajectory format	obtained from <code>gmx dump -f cg.trr > dumptrr.txt</code> .
.lmp	LAMMPS’ dump file	A text file with the information presented like LAMMPS’ output files.
.data	LAMMPS’ input data file format	A text file that can be used as input to a LAMMPS simulation.

3.1.3 Units

Note that we store everything in Gromacs’ standard units. Whenever we read in a .lmp file, we assume that it was generated using the “real” units in LAMMPS. Similarly, whenever we write a .lmp or .data file, we write it using the “real” units. Specifically, when reading in a file, times are scaled by a factor of 0.001 (fs to ps), distances are scaled by a factor of 0.1 (angstrom to nm), velocities are scaled by a factor of 100 (angstrom/fs to nm/ps), and forces are scaled by 4.184 (kcal/mol-angstrom to kJ/mol-nm).

3.2 Topology Requirements

BOCS’s **translator** tool will generate a .btp file from a dumped gromacs .tpr file. BOCS uses this dumped .tpr file to extract the number of molecules in the system. For each molecule, the dumped .tpr will contain a list of composite atoms and the types intra-molecular of interactions each atom contributes to. (e.g. bonds, angles, dihedrals, and nb exclusion rules.)

The **translator** tool extracts this information according to the format expected of a dumped .tpr for each version of Gromacs. As developers, we try to keep up with Gromacs releases and update the translator accordingly, please check release notes on Github for translator gromacs version compatibility.

In the event that the translator tool is not compatible with the current version of gromacs a user has access to, users should launch a GitHub PR reporting the bug for developers to update compatibility.

The BOCS topoology (BTP) file is human readable and can be made without the **translator** tool. The below section outlines how to manually store topology information in

the .btp file. Beyond these version restrictions on the translator, BOCS is compatible with any topology successfully translated into the .btp file.

3.3 BOCS Topology File (.btp) Format

Although the .btp file can be generated by the `translator` tool from a dumped Gromacs .tpr file, sometimes it is useful to edit the file manually rather than re-translating a new file. Specifically, if the user is not familiar with the Gromacs tools, it may be simpler to create a .btp file by hand (or generate a script to do so) rather than learning the intricacies of .tpr file generation via Gromacs.

This section provides an example .btp file and explains each directive/how it impacts the force-matching calculation. Below we outline each expected directive in the topology file before concluding with an example, we provide a concrete examples for a branch molecule in the next section.

We begin this section with an example .btp file, created for a system of 125 “branch” molecules.

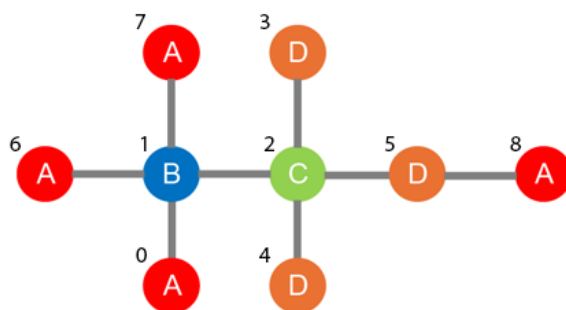


Figure 3: 1 Instance of a “Branch” molecule. Colors and letters denote BOCS atom types, numbers denote the within molecule atom index. (9 CG sites total, indexed 0-8.)

We explain each directive and its contents in the [directive paragraphs](#) below. A full file version, which contains a mixture of different molecule types and blocks, is available with the BOCS distribution under `force-matching/example_files/`.

Example .btp

```
[name] SYSTEM OF BRANCH MOLECULES

[natoms] 1125

[ffparams] 19
!ind name
0  LJ_SR
.
```

```

.
.
18 IDIHS
!atnr fudgeQQ
1 0.500000

[molblocks] 1
!moltype mol_name n_mols
0 BRANCH 125

[moltypes] 1
!ind name n_mols n_apm n_res n_cgs excl_nr excl_nra bond_nr angle_nr dih_nr b
0 BRANCH 125 9 1 9 9 75 24 52 50 0 0
[res]
BRAN
[cg]
!idx start end
0 0 0
1 1 1
2 2 2
3 3 3
4 4 4
5 5 5
6 6 6
7 7 7
8 8 8
[bond]
!idx type i j
0 1 0 1
1 2 1 2
2 3 2 3
3 3 2 4
4 3 2 5
5 1 1 6
6 1 1 7
7 4 5 8
[angle]
!idx type i j k
0 5 0 1 2
1 6 1 2 3
2 7 3 2 5
3 7 3 2 4

```

4	7	5	2	4
5	8	6	1	7
6	8	6	1	0
7	8	7	1	0
8	7	2	5	8
9	5	6	1	2
10	5	7	1	2
11	6	5	2	1
12	6	4	2	1

[dih]

!idx	type	i	j	k	l
0	9	0	1	2	3
1	10	6	1	2	3
2	11	7	1	2	3
3	12	0	1	2	4
4	13	6	1	2	4
5	14	7	1	2	4
6	15	0	1	2	5
7	16	6	1	2	5
8	17	7	1	2	5
9	18	1	2	5	8

[attypes]

!moltype_ind	at_ind	type	m	q	resind	atomname	typename	n_epa
0	0	0	59.04480	0.00000	0	A	A	8

[epa]

0	1	2	3	4	5	6	7
---	---	---	---	---	---	---	---

!moltype_ind	at_ind	type	m	q	resind	atomname	typename	n_epa
0	1	0	59.04480	0.00000	0	B	B	9

[epa]

0	1	2	3	4	5	6	7	8
---	---	---	---	---	---	---	---	---

!moltype_ind	at_ind	type	m	q	resind	atomname	typename	n_epa
0	2	0	59.04480	0.00000	0	C	C	9

[epa]

0	1	2	3	4	5	6	7	8
---	---	---	---	---	---	---	---	---

!moltype_ind	at_ind	type	m	q	resind	atomname	typename	n_epa
0	3	0	59.04480	0.00000	0	D	D	9

[epa]

0	1	2	3	4	5	6	7	8
---	---	---	---	---	---	---	---	---

!moltype_ind	at_ind	type	m	q	resind	atomname	typename	n_epa
0	4	0	59.04480	0.00000	0	D	D	9

[epa]

```

0 1 2 3 4 5 6 7 8
!moltype_ind at_ind type m q resind atomname typename n_epa
0 5 0 59.04480 0.00000 0 D D 9
[epa]
0 1 2 3 4 5 6 7 8
!moltype_ind at_ind type m q resind atomname typename n_epa
0 6 0 59.04480 0.00000 0 A A 8
[epa]
0 1 2 3 4 5 6 7
!moltype_ind at_ind type m q resind atomname typename n_epa
0 7 0 59.04480 0.00000 0 A A 8
[epa]
0 1 2 3 4 5 6 7
!moltype_ind at_ind type m q resind atomname typename n_epa
0 8 0 59.04480 0.00000 0 A A 6
[epa]
1 2 3 4 5 8

[Bocs_at_types] 4
!idx typename
0 A
1 B
2 C
3 D

```

3.3.1 [.btp directives]

[**name**] The name of the system is provided after the ‘name’ directive. In the example “SYSTEM OF BRANCH MOLECULES”. This has no input on any of the calculations within BOCS and is simply for user communication.

[**natoms**] The total number of atoms in the system is provided after the ‘natoms’ directive. At the CG resolution, this corresponds to the number of CG sites in your system, however, all-atom .btp files can be generated as well. Generally, $\sum_{\text{mol types}} n_{\text{mol type}} * n_{\text{atoms per mol type}}$.

[**ffparams**] The total number of interactions stored in the dumped tpr file is provided after the ‘ffparams’ directive. This includes all nonbonded and bonded interactions. Each interaction is then indexed and a corresponding Gromacs type is provided after each index. The content of ffparams is specific to Gromacs, it displays the number of interactions and indices Gromacs uses to map those interactions onto topological members. The last line !atnr fudgeQQ, reflects the nb mixing parameters read in by Gromacs.

The exact content of this directive has little bearing on **cgff** calculations, but can be useful for users who would like to confirm their CG topology input to gromacs has been received correctly into the .tpr via this and the dumped .tpr's human-readable format.

[**molblocks**] 'molblocks' just refers to the total number of molecule entries in the topology that list the number of molecules within the system. Thus, in most cases, the 'molblocks' directive is the number of unique types of molecules in your system. For example, users would normally list the topology for our branched 125 molecule system as in the above via e.g.

```
[molblocks] 1
!moltype  mol_name  n_mols
0          BRANCH   125
```

However, an equivalent topology listing could be:

```
[molblocks] 2
!moltype  mol_name  n_mols
0          BRANCH   120
0          BRANCH    5
```

In which the user has provided two list entries about one molecule. (nmol_blocks = 2, nmol_type = 1) BOCS can handle cases such that there are multiple instances of a given molecule, and thus molblocks does not have to equal the number of molecules types. Each molecule type is indexed from 0, given a name, and then the number of the specific molecule type is given.

[**moltypes**] The total number of site/atom types in a given molecule is provided after the 'moltypes' directive. Various parameters are extracted from the Gromacs topology and correspond to the quantities of various interactions that are found. For each molecule type, the index, type, and site indices involved in the interaction are listed. One unique entry is necessary for each unique molecule type in the system. Each molecule entry within the moltypes directive is comprised of a header followed by several sub directives detailed as below.

moltype entry header .

The moltype entry header appears directly under the molecule type directive, and appears as follows:

```
!ind name n_mols n_apm n_res n_cgs excl_nr excl_nra bond_nr angle_nr dih_nr bigdih_nr imnb_nr
0  BRANCH 125    9      1      9      9      75      24      52      50      0      0
```

This lists the molecule type index, the molecule type name, the number of atoms or “sites” comprising the molecule, the number of residues comprising the molecule, the number of charge group entries to be read, the number of per atom exclusion lists (usually = to `n_apm`), the total number of entries across exclusion lists, 3 times the number of bonds within the molecule, 4 times the number of angles within the molecule, 5 times the number of dihedrals within the molecule, 5 times the number of CHARMM style dihedrals within the molecule, and 3 times the number of interacting intramolecular (special 14) non-bonded pairs. Each number determines if a subdirective should appear following the molecule index header. For example, if the number of intramolecular nonbonded interactions is 0 (as in the above example), then the `[imnb]` sub directive is not required.

Following each `moltype` directive, the subdirectives defining the molecule will follow.

moltype sub-directives

- **[res]**
List of composite residues, reflects naming from gromacs topology, not utilized during `cgff`.
- **[cg]**
List of charge groups. This is not currently utilized by BOCS and not a required directory. Previously it would hold a group id and the indices of atoms within that group ID for the purpose of making electrostatic calculations parallelized in older MD code (Gromacs pre 2020).
- **[bond]**
This stores the list of bonded interaction instances within the topology in the format `idx type i j`. `idx` is the bond instance index. e.g. If a molecule has 6 bonds, the indices will run 0 through 5. `type` is the type index reflects whether those 6 bonds are treated differently or not in the dumped `tpr`. (e.g. all the same type, `type idx` is 0.) This type index is currently not utilized by BOCS, which sorts interactions based on site types involved, but is included for trouble shooting CG simulation input. `i` and `j` are the index of the atoms involved in that bonded interaction.
- **[angle]**
Analogous to the bond sub-directive, this stores the list of atom indices involved in an angular interaction together in `idx type i j k` format, where `j` is the central atom.
- **[dih]**
Analogous to the bond sub-directive, this stores the list of atom indices involved in an angular interaction together in `idx type i j k l` format, where `j` and `k` are the central atoms.

- **[imnb]**

analogous to the bond sub-directive, stores the list of atom indices involved in “special” pairwise intramolecular interactions. e.g. 14 interactions, or intramolecular pair interactions. Not related to nonbonded pair interactions.

[attypes] After all molecules have been listed, the attypes directive defines the characteristics of each atom and assigns them to the previously defined molecule type. i.e. the ‘attypes’ directive lists each type of atom/site found within each molecule. The mass and charge are listed along with specific descriptors describing the atom/site. Each atom type entry consists of a header respectively labeling the molecule the atom belongs to, the index of the atom type entry within that molecule, a (currently unused) type index, the mass of the atom, the charge of the atom, the index of the residue the atom belongs to, the name of the atom, the type of the atom and the number of intramolecular neighbors excluded from this atom. Currently, BOCS employs the typename to associate each atom with an interaction in the requested force-field during **cgff**.

Each atom type entry has only 1 sub-directive, **[epa]**, which contains the list of atom indices within the same molecule that an atom should NOT interact with via non-bonded pairwise interactions. (epa = exclusions per atom).

For example in the toy molecule referenced in figure 3, the exclusions list for site 0 contains all sites except for the tail end of the molecule (site 8). Likewise site 8 does not include site 0 in its exclusions list.

```
[attypes]
!moltype_ind  at_ind  type  m          q          resind  atomname  typename  n_epa
0              0       0    59.04480  0.00000      0         A         A        8
[epa]
0  1  2  3  4  5  6  7
.
.
.
!moltype_ind  at_ind  type  m          q          resind  atomname  typename  n_epa
0              8       0    59.04480  0.00000      0         A         A        6
[epa]
1  2  3  4  5  8
```

[Bocs_at_types] The total number of atom/site types in the system is provided after the ‘Bocs_at_types’ directive. Each site is then indexed and named. These typenamees do not need to be the same as the atomnames. During the **cgff** calculation, typenamees listed in this topology file are checked with the par.txt file passed by the user to make sure force-fields requested during **cgff** and topology stored in the .btp are consistent.

4 CGMap

The `cgmap` tool maps an all-atom trajectory to its coarse-grained representation. It takes as input an all-atom trajectory and a user-defined mapping topology file and generates a CG trajectory file and/or a single frame CG.gro file.

4.1 Basic Usage

Case 1: Generating a single frame CG gro file

```
cgmap -p {map.top} -f {AA.trr} -c {CG_frame.gro}
```

Case 2: Mapping a AA trajectory to its CG representation

```
cgmap -p {map.top} -f {AA.trr} -o {CG.trr} -s {CG.btp}
```

The output trajectory file format is specified by the file name extension, similar to the translator in the previous section. See Table 2 from that section for file extensions. Note that the CG.btp file is only required if you want to print the mapped trajectory in the .gro, .lmp. or .data file formats. It is **not** required to map a single frame to a .gro file, however. See Table 3 for all possible flags and their uses.

Table 3: The possible command line flags available for use with the CG map program, and what each specifies.

Flag	Type	Values	Description
-p	string	map.top	The name of the mapping topology file
-f	string	aa.trr	The name of the AA trajectory you want to map
-o	string	cg.trr	The name of the resulting CG trajectory
-c	string	cg1.gro	The name of a single CG gro file to write
-s	string	cg.btp	The name of a CG topology file. Req. for .gro format output
-h	bool		Print help info and quit

4.2 Mapping Topology File Format

The basic layout of the mapping topology file is quite similar to a simplified GROMACS topology file. Directives are contained in square braces, and semicolons are used to indicate comment lines. Columns within a directive are separated by whitespace. All directives listed below are mandatory, though depending on the system and the chosen mapping, you may have more than one of each. For the in-text example in this section, we will be assembling a mapping topology for a 1-site, center-of-mass model of water.

```
[ moleculetype ]
```

```
[ moleculetype ]
```

```
; name      n_atoms  
H2O        3
```

In the ‘moleculetype’ directive, you will give the name or abbreviation of one of the molecule types in your system followed by the number of atoms in one of those molecules. This signifies the beginning of the mapping information of that molecule type, and the next two directives must be ‘sitetypes’ and ‘atomtypes’ for that molecule, in that order.

[sitetypes]

```
[ sitetypes ] 1
; name mapping_type n_atoms
  WT      com          3
```

In the ‘sitetypes’ directive, you will define one or more sites for the current moleculetype by providing names, mapping types, and the number of atoms for each of the sites defined. The number of site types in the molecule must be provided after the ‘sitetypes’ directive on the same line. Here, we have defined a single site named ‘WT’ and specified that it will be located at the center of mass of three atoms. The currently supported mapping types include:

```
com - Maps the CG site to the center of mass of the indicated atoms
cog - Maps the CG site to the center of geometry of the indicated atoms
usr - Maps the CG site according to user-provided weights for each atom,
      which must be normalized within a site.
```

Note that the choice of mapping type here will impact what information is read from entries in the ‘atomtypes’ directive.

[atomtypes]

```
[ atomtypes ]
; index  type  site  mass/weight
   1      O    TW    15.999
   2      H    TW     1.008
   3      H    TW     1.008
```

In the ‘atomtypes’ directive, you will specify the atoms in the atomistic topology and provide which CG site they map to. The index column corresponds to the index of that atom in the corresponding atomistic topology, allowing the mapping program to identify it in the atomistic trajectory. The type column is used for internal labeling and ease of visually reading the mapping topology file. The site column contains the name of a site from the ‘sitetypes’ directive, or the word ‘none’ if the atom is not used to determine the position of a CG site. An example at the end of this section will show the use of the ‘none’ keyword in a mapping topology file for toluene. The contents of the mass/weight column depend on the choice of mapping type for the site the atom belongs to. For a ‘com’ mapping, this column will contain the mass of the atom in amu. For a ‘usr’ mapping type, this column will contain

a user-specified mapping weight. Note that the sum of these user-specified weights must sum to one for the atoms in a single site. For a ‘cog’ mapping type, the mass/weight column is omitted.

```
[ molecules ]
```

```
[ molecules ]
; name  number
  H2O    216
```

In the ‘molecules’ directive, you will specify the composition of the system. The name column will contain the name of a molecule contained in a previous ‘moleculetype’ directive, and the number column will contain the number of that type of molecule in the system.

4.2.1 Multiple molecules in a system

Many systems of interest will contain more than one type of molecule. For such systems, additional ‘moleculetypes’ directives specify the different molecule types present in the system, which each require their own ‘sitetypes’ and ‘atomtypes’ directives. All molecule types must be enumerated under the ‘molecules’ directive.

Below, we have provided an example of a mapping topology file for a more complex system, composed of 221 3-site heptane molecules and 331 3-site toluene molecules. Note that each ‘moleculetype’ directive is followed immediately by a ‘sitetypes’ directive and an ‘atomtypes’ directive corresponding to mapping information for that molecule. Additionally, the order in which the molecule types are defined in the file corresponds to the order of molecules in the atomistic trajectory - the first 221 molecules are heptane, and the last 331 molecules are toluene. This is a requirement for the calculation to proceed correctly. Finally, note that the last directive in the file is a single ‘molecules’ directive that specifies the number of both heptane and toluene molecules within the system, again in the same order as the atomistic trajectory file.

```
[ moleculetype ]
```

```
; name    n_atoms
  HEP      23
```

```
[ sitetypes ] 3
```

```
; name  mapping_type  n_atoms
  CTA    com           7
  CM     com           9
  CTB    com           7
```

```
[ atomtypes ]
```

```
; index  type  site  mass/weighting
   1      C    CTA   12.011
```

2	H	CTA	1.008
3	H	CTA	1.008
4	H	CTA	1.008
5	C	CTA	12.011
6	H	CTA	1.008
7	H	CTA	1.008
8	C	CM	12.011
9	H	CM	1.008
10	H	CM	1.008
11	C	CM	12.011
12	H	CM	1.008
13	H	CM	1.008
14	C	CM	12.011
15	H	CM	1.008
16	H	CM	1.008
17	C	CTB	12.011
18	H	CTB	1.008
19	H	CTB	1.008
20	C	CTB	12.011
21	H	CTB	1.008
22	H	CTB	1.008
23	H	CTB	1.008

```
[ moleculetype ]
; name      n_atoms
TOL        15
```

```
[ sitetypes ] 3
; name      mapping_type  n_atoms
CF          com          5
CBA         com          1
CBB         com          1
```

```
[ atomtypes ]
; index      type      site      mass/weighting
1           C          CF        12.011
2           H          CF        1.008
3           H          CF        1.008
4           H          CF        1.008
5           C          CF        12.011
6           C          none      12.011
7           H          none      1.008
```

8	C	none	12.011
9	H	none	1.008
10	C	CBA	12.011
11	H	none	1.008
12	C	CBB	12.001
13	H	none	1.008
14	C	none	12.011
15	H	none	1.008

```
[ molecules ]
; name number
HEP 221
TOL 331
```

5 CGFF

The CGFF tool derives a CG force field from a mapped AA trajectory using force matching and/or the g-YBG method.

5.1 Input Files

The main input file for the `cgff` code (which MUST be named `par.txt`) contains a list of directives and associated parameters. All other input files are dependent (either directly or indirectly) on the information placed in `par.txt`. Below is a list of all the directives currently available for specifying input options. Some directives are mandatory, while others are optional (as indicated below). The directive name must be placed inside square brackets, `[directive_name]`, followed by the appropriate options and an end directive line, `[End directive_name]`. (Examples will be given below).

5.2 List of Directives

1. Mode
2. Temperature
3. Structures
4. Site_Types
5. Inter_Types
6. Pair_Interaction
7. BondStretch

8. Angle
9. Dihedral
10. IntraMolec_NB_Pair
11. External_Potential
12. Reference_Potential
13. Perturbation_Theory
14. Eigen
15. SVD
16. TPR
17. TOP_EXCL
18. Metric_Tensor
19. Mean_Force_Decomposition
20. Calculation_Mode
21. Preconditioning
22. Solution_Method
23. Calculate_Errors
24. Reference_Options
25. TRIM
26. CHISQD
27. Regularization
28. Rescale_Forces
29. Constrain_Dihedrals
30. Iterative_gYBG
31. Frame_Weighting
32. Use_Old_Algorithm

5.3 Mandatory Directives

Mode Syntax:

[Mode] MODE

Currently, the only available mode is `MODE = "GROMACS"`. However, this directive remains as a stub for file formats that may be supported in the future. Use Gromacs `.trr` file(s) and bocs topology file(s) to obtain structural and force (if applicable) information for the calculation. In this mode, the BOCS topology file (`.btp`) will be used to set up the CG topology. So, the interactions in your `CG.btp` file must match up with those listed in `par.txt` (otherwise, the program will terminate and return errors).

Temperature Syntax:

[Temperature] T

T is the temperature at which the the trajectories were generated as a floating point number. T is used to calculate each $b_\zeta(x) = r_m k_B T (\frac{d}{dx} \bar{g}_\zeta(x) - L_\zeta(x))$ from structural information. The units of T determine the units of ϕ_ζ and b_ζ in the output (units of Kelvin correspond to standard Gromacs units).

Structures Syntax:

```
[Structures] N_Structure_Files
filename1
filename2
...
... x N_Structure_Files
...
[End Structures]
```

The default entry for this directive is to point to the input text file used to specify the trajectory (structure files) that will be used as input to develop the CG forcefield. In most cases there is 1 structure file and the directive is given by `"inp.txt."` However, each filename under the structures directive can contain a list of filenames that store the necessary structure and force information (if applicable) for the calculation. For example, in Gromacs mode the structure file contains one or more `.trr` filenames.

Here is the syntax of the input text file (`inp.txt`):

```

[Struct_Files] N_files
filename1      w1
filename2      w2
...
...   x N_Files
...
[End Struct_Files]

```

The w_i after each filename is a weighting term that will be applied to the information in that file in the FF calculation. The weights of all structure files given should sum to 1.0. If MODE = Gromacs (as described above), then each filename corresponds to a .trr file. The BOCS topology filename (.btp file) which is also required is not explicitly specified. Instead it is inferred from the .trr filename in the structure file (i.e., the .trr and .btp topology filenames must be the same). For example, if your CG trajectory file is cg.trr, the inferred CG BOCS topology file name is cg.btp. However, the default BOCS topology filename can be overwritten using the TPR directive.

Site_Types Syntax:

```

[Site_Types] N_Site_Types
Type1
Type2
...
...   x N_Site_Types
...
[End Site_Types]

```

Type_{*i*} corresponds to the name of a CG site type. These must match up with the names specified in the .btp file.

Inter_Types Syntax:

```

[Inter_Types] N_Inter_Types
inter_name1    inter_type1    basis1    basis1_options
inter_name2    inter_type2    basis2    basis2_options
...
...   x N_Inter_Types
...
[End Inter_Types]

```

inter_name: the name of a CG interaction, specified by the user. This name will appear in the naming of relevant output files.

inter_type : the type of each interaction, defined by the scalar order parameter which characterizes the interaction.

Pair_Interaction a nonbonded pair interaction

Local_Density a nonbonded local density interaction

LD_Gradient a nonbonded local density gradient interaction

External an external interaction acting in the z direction

BondStretch an intramolecular bond

Angle an intramolecular angle

Dihedral an intramolecular dihedral

IntraMolec_NB_Pair an intramolecular pair interaction

basis_type and **basis_options**: Corresponds to the basis in which to expand the given force interaction. Each basis has different options which must be specified. The options should follow the basis name as shown above, be in the order shown below, and contain only blank spaces between them.

delta represents the interaction on a grid using constant values at each gridpoint and can be applied to all **inter_types**.

dr the grid spacing as a floating point number

r_min the minimum grid value as a floating point number

r_max the maximum grid value as a floating point number

n_smooth smooth the b_ζ 's by a running average with this many points.

linear represents the interaction on a grid using linear interpolation between gridpoints and can be applied to all **inter_types**.

dr the grid spacing as a floating point number

r_min the minimum grid value as a floating point number

r_max the maximum grid value as a floating point number

n_smooth smooth the b_ζ 's by a running average with this many points.

Bspline represents the interaction on a grid using k^{th} order Bspline interpolation between gridpoints and can be applied to all **inter_types**.

dr the grid spacing as a floating point number

r_min the minimum grid value as a floating point number

r_max the maximum grid value as a floating point number

k the order of the spline ($k=4 \Rightarrow$ cubic Bspline)

n_smooth smooth the b_ζ 's by a running average with this many points.

power represents the interaction as a sum of Lennard-Jones-type functions. Only valid for Pair_Interaction types

r_max the cutoff distance for nonbonded pair interaction as a floating point number

N_powers the number of $\frac{1}{r^n}$ functions to use

n_i - the specific power for each function, from smallest to largest

harmonic represents the interaction as a harmonic function with two parameters. Only valid for BondStretch and Angle types.

RB (Ryckaert-Bellman): represents the interaction as a Ryckaert-Bellman cosine series. Only valid for Dihedral types.

TOY represents the interaction as a different cosine series. Only valid for Dihedral types.

Extra Intertype Arguments The Local_Density and LD_Gradient and External_Potential interactions require extra parameters after the “n_smooth” parameter (after the basis options are passed).

Example Syntax Structure:

```
[Inter_Types] Nintertypes
! Name Type    basis knot settings          | Extra Args!
inter_name1 Local_Density  basis1  basis1_options indicator r0 rc
inter_name2 LD_Gradient    basis2  basis2_options indicator r0 rc
inter_name3 External_Potential basis3 basis3_options force_zero zPBC
```

Required Extra Args for Local_Density and LD_Gradient

indicator function Following each many body interaction, users must pass the keyword corresponding to the weighting function $w(r, r_0, r_c)$ employed to define local densities. This weighting function should be the same for defining the local densities employed in LD and LD_Gradients that act on the same pair of site types. (e.g. $\rho_{A|B}$ must be defined the same for $U_{A|B}(\rho)$ and $U_{\nabla;A|B}(\rho)$)

The recognized weighting function keywords are:

1. shell
2. sphere
3. smooth
4. lucy
5. dpd

Exact functional forms and use cases are detailed by the LAMMPS PKG-BOCS manual located on the noid-group:lammps-BOCS2026 repository, the official LAMMPS distribution documentation and in our second manuscript releasing BOCS (Lesniewski, DeLyser, Noid (2026)).

- r0** Following the weighting function keyword, the lower cutoff, r_0 , must be specified. This is most relevant for the 2 length scale indicator functions, shell, sphere, and smooth. r_0 must be set to 0 for the lucy and dpd indicator functions.
- rc** Following the r0 setting, the cutoff value r_c is set for the indicator function passed. Note that after this length scale LD/SG forces are zero.

Required Extra Args for External_Potential Interactions

- force_zero** This parameter specifies whether or not the user would like to construct an external field in the z direction of the simulation box. If this is “0” you want the MSCG solution with the external force set to 0, while taking into account the density profile during the parameterization. (Effectively solving an over-determined system of equations.) If this is “1”, then you parameterize the external force.
- zPBC** This parameter sets the periodic boundary conditions in the z dimension for the external potential information constructed. If this is “0”, then the z direction does not have periodic boundary conditions. For example, this could be a simulation with walls capping the box in the z direction. If this is “1”, then the z direction does have periodic boundary conditions.

5.4 Dependent Directives

These directives are not strictly mandatory, but may be required by the options specified in other directives.

Pair_Interaction Syntax:

```
[Pair_Interaction] N_Pair_Types
inter_name1    type1a    type1b
inter_name2    type2a    type2b
...
... x N_Pair_Types
...
[End Pair_Interaction]
```

This directive is mandatory when there are Pair_Interaction, Local_Density, or LD_Gradient types specified under the Inter_Types directive. Each inter_name corresponds to one of the

interaction names under the Inter.Types directive. Each type corresponds to one of the CG site types under the Site.Types directive. A single interaction type may be listed more than once in order to combine several pairs of site types into a degenerate interaction type.

External_Potential Syntax:

```
[External_Potential] N_External_Types
inter_name1 type1
inter_name2 type2
...
... x N_External_Types
...
[End External_Potential]
```

This directive is mandatory when there are External interaction types specified under the Inter.Types directive. Each inter_name corresponds to one of the interaction names under the Inter.Types directive. Each type corresponds to one of the CG site types under the Site.Types directive.

BondStretch Syntax:

```
[BondStretch] N_Bond_Types
inter_name1 type1a type1b
inter_name2 type2a type2b
...
... x N_Bond_Types
...
[End BondStretch]
```

This directive is the same as the Pair_Interaction directive, except for BondStretch types. The pair of site types must define a bond type in the .btp file. Additionally, each bond type in the .btp file must be represented here. Furthermore, in the topolgy (.btp) file used, all of the CG bonds must be the same function type - either 1 (bond) or 8 (tabulated bond).

Angle Syntax:

```
[Angle] N_Angle_Types
inter_name1 type1a type1b type1c
inter_name2 type2a type2b type2c
...
```

```
... x N_Angle_Types
...
[End Angle]
```

This directive is the same as the BondStretch directive, except for Angle types. Analogous to the BondStretch function types, all of the Angle function types in the .btp file must be the same - either 1 (angle) or 8 (tabulated angle).

Dihedral Syntax:

```
[Dihedral] N_Dihedral_Types
inter_name1  type1a  type1b  type1c  type1d
inter_name2  type2a  type2b  type2c  type2d
...
... x N_Dihedral_Types
...
[End Dihedral]
```

This directive is the same as the BondStretch directive, except for Dihedral types. Analogous to the BondStretch and Angle directives, the Dihedral function types in the .btp file must all be the same - either 1 (proper dihedral) or 8 (tabulated dihedral).

IntraMolec_NB_Pair Syntax:

```
[IntraMolec_NB_Pair] N_Types
inter_name1  type1a  type1b
inter_name2  type2a  type2b
...
... x N_Types
...
[End IntraMolec_NB_Pair]
```

This directive is the same as the BondStretch directive, except for IntraMolec_NB_Pair types.

5.5 Optional Directives

These directives are completely optional. In a few instances they may affect the functionality of one another, but are never needed based on the directives already discussed.

Reference_Potential Syntax:

[Reference_Potential] ref.txt

ref.txt is the path to a file which contains the information about reference contributions which will be subtracted out during the calculation. This method is a bit out dated. We have found that the accuracy of this method is lacking, especially for bonded interactions with many large forces being subtracted from the mean force on each site. Instead, we now implement reference forces by using the -mdrerun option in gromacs to calculate a .trr file with reference contributions on each site. We then read in this trajectory and evaluate the reference contribution to the **b** vector before subtracting from the total contribution to **b**. Options for these calculations can be found under [Reference_Options].

Calculation_Mode Specifies what part(s) of the calculation to run.

Syntax:

[Calculation_Mode]
CalcMODE
[End Calculation_Mode]

CalcMODE represents the calculation mode to perform. The options are case sensitive.

FULL (default) Runs the entire calculation from beginning to end. Save state files will be printed out so additional calculations may be done without re-calculating the correlation functions.

FIRST_HALF Reads input, calculates correlation functions, prints out save state files for later calculations.

SECOND_HALF Reads input to set up system structure, reads in save state files, trims the necessary arrays, and then performs the matrix inversion according to the input options. Note that the matrix inversion is serial, so you should never submit a job with CalcMODE = "SECOND_HALF" on multiple processors.

TEST_INP Reads input, calculates the number of force field parameters and then estimates the amount of memory you will need for the first half, second half, and full calculation. The memory estimation takes into account the optional input parameters in par.txt as well as the number of parameters. This memory estimation should be updated based on the new regularization and constraint algorithms.

TPR Syntax:

```
[TPR] N_TPR_Files
filename1
filename2
...
... x N_TPR_Files
...
[End TPR]
```

Without this directive, the program expects a .btp file for each .trr file listed in the structure files ([Structures]). These .btp files must have the same name and path as the .trr files. This directive overwrites the default .btp file paths with those specified here. The number and order of the .btp files must match up exactly with the .trr files. The program will give an error message if you input the wrong number of files, but there is nothing to check that the order is correct, so be careful! Note: we keep the legacy “TPR” name for this directive, even though we now require .btp files. .tpr files will not work.

TOP_EXCL This directive allows the user to pass in a .btp file to be used for pair exclusions only. This is useful for passing in a bond reference force without force-matching the corresponding interaction.

Syntax:

```
[TOP_EXCL]
filename.btp
[End TOP_EXCL]
```

filename.btp denotes the name of the .btp file which will be used for calculating pair exclusions. Without this directive, the usual .btp files will be used instead.

Solution_Method This directive specifies the solution method for solving the system of linear equations.

Syntax:

```
[Solution_Method]
SOLN_METH
[End Solution_Method]
```

SOLN_METH specifies the solution method to be used. The input is case sensitive

SVD (default) Solves the matrix inversion using single value decomposition.

CHOLSKY Solves the matrix inversion using CHOLSKY decomposition (for symmetric positive-definite matrices). This method utilizes packed storage of the symmetric matrix in order to save memory.

UU Solves the matrix problem using UU-type decomposition while also utilizing the packed storage form of the matrix.

LU Solves the matrix problem using general LU decomposition. Consequently, simple preconditioning can be performed on the matrix (see [Preconditioning]).

SVD This directive provides additional information about the single value decomposition calculation.

Syntax:

```
[SVD]
rcond
flag_printSV
flag_printvecs
flag_solve
[End SVD]
```

rcond a floating point value in decimal form. Each inverse singular value below this number will be set to zero before calculating the solution vector. This is only applicable when **flag_solve** is “YES”.

flag_printSV “YES” or “NO”. The singular values will be explicitly calculated and printed out if **flag_printSV** is “YES”.

flag_printvecs “YES” or “NO”. The eigenvectors corresponding to each singular value will be explicitly calculated and printed out if **flag_printvecs** is “YES”.

flag_solve “YES” or “NO”. If **flag_solve** is “YES”, the matrix problem is solved using singular value decomposition. This flag is equivalent to specifying the solution method “SVD” in [Solution_Method] (and specifying this directive will overwrite whatever option is specified under [Solution_Method]).

Error_Estimates If this directive is present, error estimates will be calculated for the CG force field obtained from the method chosen in [Solution_Method]. The inverse condition number will be approximated from the corresponding decomposition of **G** (Note: You can calculate the condition number more precisely with [SVD] or [Eigen]). Iterative refinement will also be used to estimate the forward and backward errors.

Syntax:

```
[Error_Estimates]
flag_Equil
[End Error_Estimates]
```

flag_Equil “YES” or “NO”. The matrix will be equilibrated (i.e., preconditioned) by LAPACK if flag_Equil is YES.

Preconditioning This directive applies to calculations with SOLN_METH = LU. Additionally, if [Constrain_Dihedrals], or [SVD] is specified, then the option under [SOLN_METH] is overwritten such that this directive is applicable. Generally, RPC_type = LPC_type = “dimless” with flag_normb = flag_normphi = “NO” yields the lowest condition number.

Syntax:

```
[Preconditioning]
RPC_type
flag_normb
LPC_type
flag_normphi
[End Preconditioning]
```

RPC_type specifies the right preconditioning method

NO Do not precondition $\mathbf{G}$.

dimless Precondition such that $\mathbf{G}$ is dimensionless. For this to work, “dimless” must be chosen for both RPC_type and LPC_type. To use the rcond option under [SVD], this option must be used.

colnorm divide each column of $\mathbf{G}$ by the norm of that column

MTvar divide each column of $\mathbf{G}$ by the sum of the elements in the corresponding column of $var(\mathbf{G})$

flag_normb “YES” or “NO”. “YES” rescales the preconditioning terms such that b retains the same norm before and after preconditioning.

LPC_type specifies the left preconditioning method

NO Do not precondition $\mathbf{G}$.

dimless dimless: Precondition such that $\mathbf{G}$ is dimensionless. For this to work, “dimless” must be chosen for both RPC_type and LPC_type. To use the rcond option under [SVD], this option must be used.

rowmax divide each row of $\mathbf{G}$ by the absolute value of the max of the elements in the row

bvar divide each column of $\mathbf{G}$ by the sum of the elements $var(b)$

flag_normphi “YES” or “NO”. “YES” rescales the preconditioning terms such that ϕ retains the same norm before and after preconditioning.

Iterative_Inversion This directive applies a matrix perturbation technique to iteratively calculate the solution to the force-matching equations.

Syntax:

```
[Iterative_Inversion]
N_PT
spacing
flag_MMOTF_SEP
flag_eigen
N_eigen
[End Iterative_Inversion]
```

N_PT the number of perturbation steps to perform.

spacing the number of perturbation steps between printing of the solution.

flag_MMOTF_SEP “YES” or “NO”. If “YES”, the solution at each (printed) step will be multiplied by the full metric tensor, yielding an estimate of the structural correlation functions determined by the present force field.

flag_eigen “YES” or “NO”. If “YES”, the eigenvalues and eigenvectors of the current approximation to the metric tensor will be calculated for each (printed) step.

N_eigen determines the number of eigenvectors to print out from each side of the spectrum (i.e., $N_eigen = 10$ will print the ten eigenvectors corresponding to the lowest eigenvalues and the ten eigenvectors corresponding to the highest eigenvalues).

Eigen This directive allows the user to calculate and print the eigenvalues/eigenvectors of the metric tensor.

Syntax:

```
[Eigen]
N_Eigen
flag_fnbnMn
spacing_top
spacing_bottom
flag_Gbar
flag_norm
[End Eigen]
```

N_Eigen determines the number of eigenvectors to print out from each side of the spectrum (See N_eigen under [Iterative_Inversion]).

flag_fnbnMn “YES” or “NO”. If “YES”, the eigenvector representation of the forces, b’s, and the metric tensor will be calculated for various sets of eigenvectors (according to the options below).

spacing_top A floating point number that determines the grouping of eigenvectors from the “top” (highest eigenvalues) for calculating the representation of the forces, b’s, and the metric tensor.

spacing_bottom A floating point number that determines the grouping of eigenvectors from the “bottom”.

flag_Gbar “YES” or “NO”. If “YES”, the eigenvalues/eigenvectors will be calculated for $\bar{\mathbf{G}}$.

flag_norm “YES” or “NO”. This flag controls whether or not the metric tensor is normalized by $\frac{1}{r_{\zeta'}(x)^2 r_{\zeta'}(x')^2}$ (see [Metric_Tensor]) before calculating the eigenvalues/eigenvectors.

Metric_Tensor This directive is concerned with the calculation and print options of the metric tensor and related quantities.

Syntax:

```
[Metric_Tensor]
flag_print
flag_norm
flag_Mcnt
[End Metric_Tensor]
```

flag_print “YES” or “NO”. This flag controls whether or not the metric tensor blocks are printed out for analysis purposes. (see [Rudzinski, Noid JPCB 2012](#)).

flag_norm “YES” or “NO”. This flag controls whether or not the metric tensor blocks (printed out for analysis purposes) are normalized by $\frac{1}{r_{\zeta}(x)^2 r_{\zeta'}(x')^2}$ (see [Rudzinski, Noid JPCB 2012](#)).

flag_Mcnt “YES” or “NO”. This flag determines whether or not $\mathbf{P}_{\zeta\zeta'}(x, x')$ is calculated or not. Note that flag_Mcnt = “NO” will probably decrease the amount of memory needed for the calculation (See CalcMODE = TEST_INP).

Mean_Force_Decomposition If this directive is provided in par.txt, the mean force will be decomposed into 2- and 3-body contributions for each interaction type in addition to the normal force field output.

Syntax:

```
[Mean_Force_Decomposition]
[End Mean_Force_Decomposition]
```

Reference_Options This directive has options for calculating or reading in a reference *b* vector, bref.

Syntax:

```
[Reference_Options]
flag_calcbref
flag_readbref
flag_splitfiles
flag_breftrr
breftrr_fnm
[End Reference_Options]
```

flag_calcbref “YES” or “NO”. If “YES”, the triple loop will be skipped and the calculation will be terminated once bref is calculated and printed to bref.dat

flag_readbref “YES” or “NO”. If “YES”, the calculation will proceed as normal, but the values in bref.dat will be subtracted from the calculated *b*’s. Note that this will replace subtraction of the normal bref if a reference potential is supplied. Also, if flag_calcbref = “YES”, this option does nothing.

flag_splitfiles “YES” or “NO”. If “YES”, each processor will handle a separate file (instead of splitting up the frames in each file. This is more efficient than the normal way, but requires that you split up your trajectory file by hand. Note that for nproc should divide nfiles. This option was motivated by the situation where you have an extremely large trajectory file and the interactions you are parameterizing are contained only in a subset of the molecules in the system (with reference potentials for the other molecules). In this case, one could precompute bref with flag_calcbref = “YES” and flag_splitfiles = “YES” for maximum efficiency.

flag_breftrr “YES” or “NO”. If “YES”, reference forces will be taken from a trr file named breftrr_fnm.

breftrr_fnm The file containing reference forces for flag_breftrr. This file should be generated with mdrun -rerun with the reference topology.

TRIM This directive has options for trimming the basis vectors that are rarely sampled. $FE \approx 0.005 - 0.05$ works well in practice, depending on the system. The details of this trimming are explained in detail in the SI section of [Rudzinski, Noid JPCB 2014](#).

Syntax:

```
[TRIM]
FE
[End TRIM]
```

FE the sampling cutoff for trimming rarely sampled vectors, as a decimal % of the sampling expected per bin in a uniformly sampled interaction

CHISQD This directive allows for the calculation of χ^2 for an arbitrary force vector. You can input a force vector contained in the file named `force_nm`. This vector should be consistent with the dimensions of the problem from the normal calculation and the save state input files.

Syntax:

```
[CHISQD]
force_nm
[End CHISQD]
```

force_nm The location of a file containing a complete force vector

Constrain_Dih This directive allows one to constrain the coefficients of each dihedral interaction to sum to 0. This requires that the dihedral force integrates to zero, which is necessary for the dihedral potential to be periodic. This directive is only useful for dihedral force functions represented with a tabulated basis and which sample all grids between -180 and 180 during the reference simulation.

Syntax:

```
[Constrain_Dih]
Nmax
tau
L
dL
[End Constrain_Dih]
```

Nmax specifies the maximum number of iterations in the constraint algorithm.

tau specifies the constraint parameter τ as a floating point number

L specifies the tolerance as a floating point number

dL specifies the change in tolerance during each iteration as a floating point number

Iterative_gYBG This directive has options for performing the iter-gYBG method as described in Ruzinski, Noid JPCB 2014.

Syntax:

```
[Iterative_gYBG]
flag_AAM2
AAM2_fnm
flag_bsolnerr
[End Iterative_gYBG]
```

flag_AAM2 “YES” or “NO”. “YES” allows the user to provide a file with the 2-body contributions to the metric tensor from the AA trajectory. This is necessary for the modified iter-gYBG method proposed by Ruzinski, Noid JPCB 2014. “NO” turns off this directive.

AAM2_fnm the name of the file with the AA 2-body contributions when **flag_AAM2** = “YES”.

flag_bsolnerr “YES” or “NO”. “YES” prints out the error in the projected force functions from consecutive iterations. With this option, the force files from the previous iterations must be provided.

Frame_Weighting This directive should be used for simulations run in the NPT ensemble.

Syntax:

```
[Frame_Weighting]
NPT
[End Frame_Weighting]
```

Force matching for the NPT ensemble was derived in scaled coordinates. As a result, each frame should be weighted by $V^{2/3}$ when optimizing the CG interaction potential. See [Dunn and Noid JCP 2015](#) for a more complete discussion of how this factor arises. Note: this directive should not be included for simulations run in the NVT ensemble.

Use_Old_Algorithm This directive should be specified to use the older algorithm for computing the G matrix.

[Use_Old_Algorithm]

The current algorithm is much faster. Furthermore, there are newer interaction types only supported when using the newer algorithm (local density and external interactions). The current algorithm only requires a loop over pairs of particles, whereas the older algorithm requires a loop over triples of particles. We highly recommend using the current algorithm instead of the old algorithm. Note, this directive does not require the inclusion of a corresponding end directive like other directives.

5.6 Example par.txt and inp.txt files

As an illustrative example, we present the par.txt and inp.txt files from two different calculations. The first of these is for a calculation to recover the potentials in a simple Lennard-Jones fluid. This system has a single type of nonbonded pair interaction between LJ sites, and no intramolecular interactions. Note that in both the par.txt and inp.txt files, lines beginning with a ‘!’ character are comment lines and are not read by the **cgff** tool.

Lennard-Jones: par.txt

```
!# input file
[Mode] GROMACS

[Temperature] 298

[Structures] 1
inp.txt
[End Structures]

[Site_Types] 1
LJ
[End Site_Types]

[Inter_Types] 1
!inter_name inter_type basis dr R_min R_max n_smooth
LJLJ Pair_Interaction delta 0.004 0.000 1.200 0
[End Inter_Types]

[Pair_Interaction] 1
! inter_name type1 type2
LJLJ LJ LJ
[End Pair_Interaction]
```

```
[TPR] 1
../LJ.btp
[End TPR]
```

Lennard-Jones: inp.txt

```
! comment line
[Struct_Files] 1
../LJ.trr 1.0
[End Struct_Files]
```

The next example is for an extended ensemble calculation, over three alkane trajectories: butane, heptane, and decane. These files correspond to the calculation performed in the original code release paper (Dunn et. al (2018)).

Extended Ensemble Alkanes: par.txt

```
!# input file
[Mode] GROMACS
```

```
[Temperature] 298.0
```

```
[Structures] 1
inp.txt
[End Structures]
```

```
[Site_Types] 2
CT
CM
[End Site_Types]
```

```
[Inter_Types] 8
```

!inter_name	inter_type	basis	dr	R_min	R_max	k	n_smooth
CTCT	Pair_Interaction	Bspline	0.02	0.000	1.400	4	0
CMCM	Pair_Interaction	Bspline	0.02	0.000	1.400	4	0
CTCM	Pair_Interaction	Bspline	0.02	0.000	1.400	4	0
!inter_name	inter_type	basis	dr	R_min	R_max	n_smooth	
Bond_CT-CM	BondStretch	linear	0.001	0.000	0.400	0	
Bond_CT-CT	BondStretch	linear	0.001	0.000	0.400	0	
Bond_CM-CM	BondStretch	linear	0.001	0.000	0.500	0	
Angle_CT-CM-CT	Angle	linear	0.500	0.000	180.0	0	
Angle_CT-CM-CM	Angle	linear	0.500	0.000	180.0	0	

```

[End Inter_Types]

[Pair_Interaction] 3
! inter_name      type1  type2
  CTCT            CT    CTF
  CMCM            CM    CM
  CTCM            CT    CM
[End Pair_Interaction]

[BondStretch]      3
!inter_name      site1      site2
  Bond_CT-CM      CT          CM
  Bond_CT-CT      CT          CT
  Bond_CM-CM      CM          CM
[End BondStretch]

[Angle]            2
!site1      site2      site3      type
  Angle_CT-CM-CT      CT          CM          CT
  Angle_CT-CM-CM      CT          CM          CM
[End Angle]

[TPR] 3
../cgbutane.btp
../cgheptane.btp
../cgdecane.btp
[End TPR]

```

Extended Ensemble Alkanes: inp.txt

```

[Struct_Files] 3
map_butane/cgbutane.trr 0.33
map_heptane/cgheptane.trr 0.33
map_decane/cgdecane.trr 0.33
[End Struct_Files]

```

5.7 Output Files

This section details different files BOCS outputs, what they mean, and when to expect them.

Each of these subsections are written with the expectation that the user understands the force-matching algorithm at a “coarse” level. Particularly, before reading the output descriptions the user should understand that the force-matching algorithm consists of two parts:

1. Reading each frame of an atomistic trajectory to collect positional/force information and fill the $\mathbf{G}$ matrix and $\mathbf{b}$ vectors. We sometimes refer to this as the FIRST_HALF of the calculation in e.g. the CalcMode directive of the par file. This part of the calculation is typically parallelized by having processor number N accumulate data from every N th frame.
2. Solving the total accumulated system of linear equations $\mathbf{G}\phi = \mathbf{b}$. This is the serial part of the calculation (the SECOND_HALF) where we invert $\mathbf{G}$ according to the specified SOLN_METH

Below is a table of important cgff output. Specifically we list the filename patterns in wildcard format, such that $*$ will be replaced by custom interaction names and BOCS site types filled in by the user when constructing par.txt. (e.g. a for a SOLVENT-SOLVENT pair interaction, a file like f_forces.Pair_SOL_SOL.dat fits in the pattern f_forces.*.dat). We denote filenames in this format, along with when in the 2 part calculation they are typically written given input conditions. We denote how to read files that are not columns of x y data in the sections below.

Filename pattern	When Out-put	Contents
summarize_input.txt	$\diamond$	User's basis choices read from par.txt. e.g. interactions requested and solver options, along with the grid indexing that will be assigned in the system of equations as a consequence
dump_my_mols.txt	$\dagger$	Topology read in from user's passed .btp file
log.txt	$\triangle, \square$	notes which bins are trimmed for setting up linear equations, prints LAPACK numerical warnings/condition number after solving
out.total.txt	$\square$	prints χ^2 and decomposition into interactions. Also lists parameters solved for when e.g. a harmonic basis is used
save.*.dat files	$\triangle$	Structures saved for re-starting linear equations solver after trajectory is read. Table 5 contains summaries for files that are useful for analysis outside of this restart.
b_forces.*.dat	$\triangle + (\text{force})$	mapped AA forces projected onto the * component of the CG force-field (b in the system of equations)
b_struct.*.dat	$\triangle$	The b vector in the system of equations as calculated from mapped atomistic structures via the g-YGB framework
g_cnt_coeff.*.dat	$\triangle + (\text{bspline})$	Raw count of samples utilized in solving for each coefficient divided by the bin spacing dx
g_cnt.*.dat	$\triangle$	basis interpolation of sampling on each coefficient
g.*.dat	$\triangle$	$\propto \bar{g}$ from the g-YBG equations
L.*.dat	$\triangle$	$\propto L$ from the g-YBG equations (interpolated between coefficients)
L_coeff.*.dat	$\triangle$	$\propto L$ from the g-YBG equations (evaluated at each coefficient)
f_forces_coeff.*.dat	$\square + (\text{bspline})$	The spline coefficients ϕ BOCS solved for using forces in the force-matching normal equations. These are used to produce the f_forces.*.dat output
f_struct_coeff.*.dat	$\square + (\text{bspline})$	The spline coefficients ϕ BOCS solved for using the g-YGB framework of the normal equations. These are used to produce the f_struct.*.dat output
f_struct.*.dat	$\square$	these are the force functions BOCS has calculated from the mapped atomistic structures via the g-YGB framework
f_forces.*.dat	$\square + (\text{force})$	these are the force functions BOCS has calculated from the mapped atomistic forces

Table 4: BOCS cgff output files, expected conditions for output and summary of contents.

Key for timeline symbols (in order of process):

$\diamond$ = after par.txt is read

$\dagger$ = while top is read

$\triangle$ = after trajectory processing

$\square$ = after linear equations solve.

Key for input dependent symbols:

(force) indicates files only produced when input trajectory has force data

(bspline) denotes files only produced when input force-field requested in par.txt employs a

Bspline basis representation

Save file	Contents
save.Nf.dat	prints total number of frames processed
save.M.dat	prints the non-self contributions to the G matrix ($\lambda \neq \lambda'$) [before statistical trimming, in long packed form]
save.M2.dat	prints the self contributions to the G matrix ($\lambda = \lambda'$) [before statistical trimming, in long packed form]
save.G.dat	the total G matrix (M+M2) after statistical trimming (not required for second half restart, is created from M/M2) [In 3 column D D' G form]
save.FF.dat	$\chi^2[0]$ saved from atomistic forces

Table 5: Useful save state files for data analysis

5.7.1 summarize_input.txt

summarize_input.txt prints a summary of the basis set parameters (ϕ_d) that the user requested in par.txt. This file allows one to coordinate the indexes of the G-matrix printed in log.txt with physical parameter ranges sampled.

summarize_input.txt typically contains 3 sections. The first section summarizes the thermodynamic temperature input, the total number of coefficients requested via par.txt, the number/paths of trajectory files passed and the site-types read from the topology:

Mode: GROMACS

Temperature: 350.000000

N_coeff: 7578

N_structures: 1

Structure: 1 Weight: 1.000000 Name: ../../ldcalc/LJ_w_self.btj

N_Site_Types: 4

Site_Type: 1 name: A

```

Site_Type: 2      name: B
Site_Type: 3      name: C
Site_Type: 4      name: D

```

```

*****

```

The second section contains the list of the interactions the user passed into par.txt, along with details concerning the basis representation for this interaction. e.g.

```

N_Inter_Types: 53
Inter_Type: 1
Inter_Name: Pair_A_A   Inter_Type: Pair_Interaction
Basis: Bspline   i_basis: 6
dr: 0.010000   R_min: -0.040000   R_max: 1.540000
N_pts: 159   N_coeff: 159   n_smooth: 0   i_0: 0
kspline: 4
usR_min: 0.000000   usR_max:   1.500000

```

```

Inter_Type: 2
Inter_Name: Pair_B_B   Inter_Type: Pair_Interaction
Basis: Bspline   i_basis: 6
dr: 0.010000   R_min: -0.040000   R_max: 1.540000
N_pts: 159   N_coeff: 159   n_smooth: 0   i_0: 159
kspline: 4
usR_min: 0.000000   usR_max:   1.500000

```

In the above parameters for 2/53 of the requested interactions are shown. Both are pair interactions represented with a 4th order (cubic) Bspline. The user requested that BOCS attempt to solve for forces along the interval $[r=0.00, 1.5]$, so BOCS pads the region requested ($r = [-0.04, 1.54]$) such that the the nonzero parameters ϕ_d that contribute to each bound will be solved for. Thus the number of parameters BOCS solves for each of the pair interactions is $159 = ((1.54 - 0.04)/ 0.01) + 1$. This means that the first interaction (Pair_A_A) will have indices $D = [0, 158]$ in the untrimmed G matrix, while the second interaction (Pair_B_B) will have indices $D = [159, 317]$ in the untrimmed G-matrix. For convenience, summarize_input.txt prints this first index (e.g. $i_0 = 0$, or 159) and the number of coefficients associated such that each interaction has a block of the normal equation's G-matrix. **These indices will change in the final system of equations** according to the specified trimming procedure, **see the information in log.txt for final indices**.

This second section will also re-print this information binned by specific interaction type (e.g. Pair, Bond, Angle etc.)

Finally, summarize_input.txt will conclude with the list of any topology files passed, along with any relevant settings passed through optional cgff headers. For example if the [TRIM] directive is used, this section might look like:


```
printing TPR filenames ...
```

```
N_TPR: 1
```

```
TPR[0]: box.btp
```

```
*****
```

```
printing Calculation_Mode parameters ...
```

```
CalcMODE: FULL
```

```
*****
```

```
printing Trim options ...
```

```
FE: 0.000001
```

5.7.2 log.txt

log.txt reports the progress of the second half of the force-matching calculation, which consists of two main portions.

- 1) Trimming the G matrix
- 2) Solving $G\phi = b$ with the trimmed quantities.

log.txt starts with total number of frames read from the trajectory and a flag for entering the solver function as in the example below:

```
Read 1501 frames for ../training_data/training_traj.trr.
```

```
In get_b_struct_and_b_forces.
```

The next lines of log.txt flag the trimming process of the BOCS code. e.g.

```
In trim_Mb.
```

This trimming process reports the amount of sampling each coefficient in half G contains before deciding whether to drop that coefficient from the system of equations according to the settings passed in the cgff [TRIM] directive, or default settings. In the code this is done by copying over the elements of the G -matrix that were sampled to a new trimmed G -matrix. log.txt thus reports old untrimmed indices and new “trimmed” indices.

e.g. When a coefficient is trimmed, the log file will contain a line like:

```
i0: 0 i0_inter: 0 sys->g_cnt[ 0]: 0.000000  
looping.
```

where $i0$, is the initial coefficient index and can be inferred from summarize_input.txt. “looping” denotes that this index was not copied to the new trimmed matrix. Conversely, if the element is copied into this new trimmed matrix, the log file will print something like:

```
i0: 506   i0_inter: 506   sys->g_cnt[ 506]: 0.014799
i1: 0     i1_inter: 0
```

where i1 denotes the index of the coefficient in the new trimmed matrix.

By default BOCS will trim coefficients with sampling equal to 0 and $0.001 \times$ the expected uniform sampling equivalent.

When trimming is complete and the BOCS enters the function that calls LAPACK the log.txt will signal this and print the functions called by BOCS for solving the equations. e.g.

```
Exiting trim_Mb.
```

```
In get_phi_struct_and_phi_forces.
```

Finally, log.txt will output the LAPACK solver option used and corresponding Rank for each normal system of equation solved. (e.g. the structure-based and the force-based equations)

```
In get_phi_forces.
```

```
In solv_lin_eqns.
```

```
Inverting the matrix using Singular Value Decomposition in default solver
```

```
dgelss_() took 0.000000 seconds or 0.000000 minutes or 0.000000 hours
```

```
Rank(G) = 30
```

```
In get_phi_forces.
```

```
In get_phi_struct.
```

```
In solv_lin_eqns.
```

```
Inverting the matrix using Singular Value Decomposition in default solver
```

```
dgelss_() took 0.000000 seconds or 0.000000 minutes or 0.000000 hours
```

```
Rank(G) = 30
```

The rank can be compared to the final i1 dimension of the trimmed matrix to determine if the system solved has a corresponding null space. Solver options and statistics output can also be modified using the cgff [SVD] and [SOLN_METHOD] directives.

5.7.3 out.total.txt

This file prints $\chi^2(\phi)$ along with its decomposition into $\chi^2[\mathbf{F}^0]$ and $\frac{1}{3N} \|\mathbf{F}^0 - \mathbf{F}(\phi)\|^2$. See equations 72-76 of W.G.N's 2013 book chapter (Systematic Methods for Structuarlly Consistent Coarse-Grained Models) for details. This file prints the total components of the decomposition as well as how each interaction contributes.

Below is an example excerpt from out.total.txt with explanatory comments indicated by #
Note that the comments below are referencing [W.G.N \(2013\) Methods Mol. Biol. 924 487-531.](#)

```
*****
ff = 555333.789878      # chi2_0 (the first term in rhs of eq. 76 in book chapter)
-2bphi = -1110667.532649 # Second term in rhs of eq. 76 in book chapter
phiGphi = 555333.766324 # Third term in rhs of eq. 76 in book chapter
Chi2 = 0.023554 # The total, the lhs of eq. 76 in book chapter, the Chi2 we min
*****
```

6 Tables

The tables tool creates GROMACS table files for non-bonded and bonded interactions. The bonded interactions include bond-stretches, bond-angles, and bond-dihedrals. The input consists of a file that contains a tabulated force. The independent variable (e.g. distance or angle) must be evenly spaced. The program will quit if this condition is not met. Additionally, for angle and dihedral forces the end points must be consistent with GROMACS requirements which are specified below in the appropriate section. The tabulated forces should already be smooth and any unphysical data should be removed before using this program. Additional parameters are required for certain interaction types as indicated below. The type of interaction is specified at the command line.

6.1 Non-bonded Interactions

For non-bonded interactions, the maximum distance for writing the table is required as input. The force is linearly extrapolated into the hard-core region with a slope determined from the first two points in the input file. It is assumed that the force already converges to zero at the cutoff. The force is saved as zero for points not listed but within the maximum distance specified on the command line. Not that this tool does not work with any Local Density or External Field interactions.

6.2 Bonded Interactions

Bond Stretch The minimum and maximum distance for the bond stretch must be specified at the command line. These should include any possible bond length that may be sampled in the simulation. The force curve will be extrapolated in both directions from the minimum and maximum distances in the input file until the minimum and maximum distance from the command line is reached. The extrapolation is linear with a slope that is specified at the command line. The user should make wise choices. The intent is that outside the range given in the input file, the potential becomes harmonic and should push the bond back to distances given in the input file. The minimum and maximum distances specified at the command line should include any possible bond lengths sampled during a simulation. BOCS will complain if this is not satisfied.

Bond Angle Only the slope for linearly extrapolating the input force is required at the command line. GROMACS requires that the force begin and end at 0.0 degrees and 180.0 degrees, respectively. The program will complain if the angle variable cannot be extended in either direction with the grid spacing and match these end-point conditions. It should be noted that GROMACS expects a force taken as the derivative in degrees of the specified potential, so the input should be provided appropriately. The program will extrapolate linearly with the specified slope from the minimum and maximum input angles to 0.0 and 180.0 degrees. As with the bond stretch, this becomes a harmonic potential at the extremes to push the angle back to the values specified in the input file.

Bond Dihedral No special command line parameters are specified for the dihedral interactions. However, the input must have first and last angles of -180.0 and 180.0, respectively. Also, as with angle, everything is specified in terms of degrees.

6.3 Usage

`tables [infile] [outfile] [type] [basis] (rmin) (rmax)`

Infile a two-column file containing evenly spaced distance (or angle) and force pairs

Outfile the name of the destination table file

Type Specifies the type of the interaction.

nb nonbonded interactions (requires rmin argument)

bond bond stretches (requires rmin, rmax argument)

angle angle bending interactions

dihedral dihedral interactions

Basis The basis type used for force-matching the interaction

delta

linear

Bspline

6.4 Examples

Nonbonded Lennard-Jones interaction with a linear basis:

```
tables LJLJ_force.dat table_LJLJ.xvg nb linear 6.00
```

CT-CT bond stretch with a delta basis:

```
tables CTCT_bond_force.dat table_b0.xvg bond delta 0.2 0.5
```

CT-CM-CT angle bending with a delta basis:

```
tables CTCMCT_angle_force.dat table_a0.xvg angle delta
```

CA-CB-CC-CD dihedral torsion with a Bspline basis:

```
tables CA-CB-CC-CD_dih_force.dat table_d0.xvg dihedral Bspline
```

6.5 Converting to LAMMPS tables

The files described herein are stored in `BOCS/force-matching/scripts/`.

6.5.1 Pair/Intramolecular Interactions

In order to facilitate use of these CG force fields with LAMMPS, a `translate_tables.py` python script is included in the `scripts` folder for conversion of GROMACS table files to LAMMPS table files. The output assumes the use of ‘real’ units in the LAMMPS simulation. This usage format for this script is as follows:

```
convert_tables.py [gromacs_tables] [table_type] [lammmps_table] [interaction_name]
```

gromacs_table The location of a GROMACS table file to be converted

table_type The type of interaction contained in the table file, selected from the following:

nb specifies a nonbonded interaction table

bond specifies a bond stretch table

angle specifies an angle table

dih specifies a dihedral table

lammmps_table The name of the LAMMPS table to be written

interaction_name The name of the interaction within the LAMMPS table file. This identifies the specific interaction to LAMMPS, in the event that you want to have multiple interactions within a single file.

6.5.2 Local Density Potentials, Square Gradient Interactions

For local density and square gradients of the local density interactions, we additionally provide 3 more scripts for the purpose of quickly turning BOCS output into LAMMPS input. Respectively:

- #1) `SG_table_prep_from_spline.py <f_forces_coeff.LD_Gradient_X.dat> <lammmps_table> <deg>`
- #2) `LD_table_prep_from_spline.py <f_forces_coeff.Local_Density_X.dat> <lammmps_table> <d>`
- #3) `quick_LD_prep.py <f_forces.Local_Density_X.dat>`

All scripts provided are simple scripts written for Python 3 and convert between BOCS output units to LAMMPS “real” input units. Scripts #1 and #2 are for converting BOCS B spline output to the appropriate table in LAMMPS. Script #3 is provided as part of the 1-site Local Density tutorial and is explained in section 9.4.

Script #1 and #2 accept as arguments the spline coefficient file, the user-desired output name and the degree of the spline representation for the passed coefficients. Each script then determines the value of the U and F functions stored as input for the LAMMPS table on the same spline grid used for solving the coefficients. (e.g. these scripts determines two columns of data, respectively U_ρ/U_∇ and F_ρ/F_∇ , with the appropriate dimensional conversions for LAMMPS.)

These scripts also extrapolate the LD function passed beyond the initially sampled grid provided, such that simulations will not exit with a table read error when the CG model samples a slightly different local density distribution than the AA model. Within these scripts this is done with a simple extrapolation, taking the last coefficient passed and applying that spline polynomial across the padded domain added. See the differences between this function’s output and `f_forces.X.dat` to visualize this extrapolation.

More complicated extrapolation choices (e.g. Gaussian fits to the LD distribution/ more global polynomial fits to the edges) are detailed in the SM of previous works. (see [Szukalo, Noid J Chem Phys 2023](#) for Gaussian fitting and [Lesniewski, Remsing, Noid J Chem Phys 2026](#) for polynomial fitting).

7 Pressure-Matching

7.1 Introduction

This document outlines the different filetypes that are used in the current pressure-matching implementation.

7.2 Input file: *.ini

The name of this file is specified at runtime as a command line argument of the form:

```
p_match config.ini
```

For the purposes of this document we will call the input file ‘config.ini’, but you are free to name it anything you would like. This feature is included so that multiple configuration files can coexist within one directory. Note that you would need to specify different output filenames for each configuration file to avoid overwriting your results. The settings in this file are all specified in the format

```
setting: value
```

Note that the colon is a required part of the formatting. The different settings and acceptable values are listed below. The order in which the settings are listed in the file does not matter.

7.2.1 Universal settings

These settings are required for every pressure matching calculation. Depending on the values chosen for these settings, more settings may be invoked. In cases where this occurs, the extra settings are listed under the value of the original setting that invokes them.

Pressure_units

md Pressures are described in kJ per (mol*nm³), the internal units of pressure in Gromacs

bar Pressures are described in bar, the output units of pressure in Gromacs

Volume_units

nm3 Volumes are described in nm³, the internal and output units of volume in Gromacs

liter Volumes are described in liters

Energy_units

kJ_per_mol Energies are described in kJ/mol, the internal and output units of energy in Gromacs

Basis_type

das_andersen Represents the pressure correction as a Taylor series expansion as detailed in [MSCG V by Das and Andersen](#). Requires the following options:

N_pres_coeff The number of terms in the Taylor series to use

delta Represents the pressure correction on a decoupled grid. Invokes the following options:

vmax The high end of the volume grid

vmin The low end of the volume grid

dv The grid spacing

frac_cutoff The minimum fraction of a uniformly sampled bin to accept as sampled

Iterative

0 Performs a calculation with a 1:1 mapping of the AA and CG pressures

1 Performs a calculation with different AA and CG pressure-volume trajectories. Invokes basis-type-specific options in the `das_andersen` basis type, and the following output options:

CG_Q_output - Location of the correlation matrix Q for the for of the CG data

AA_g_cnt_output - Location of the sampling distribution for the AA data

CG_g_cnt_output - Location of the sampling distribution for the CG data

Use_reference_Fv

- 0** Do not use a reference pressure correction.
- 1** Use a reference pressure correction that will be added to that calculated for the pressure-volume trajectories given as input. Requires the following options:
 - ref_Fv_file** Location of the file containing the reference pressure correction. The format of this file depends on the basis type.

Error_estimate

- 0** Solve the set of linear equations using dgesv from LAPACK.
- 1** Solve the set of linear equations using dgesvx from LAPACK, which provides an estimate of the forward and backward error of the fit, as well as the condition number of the problem.

N_atoms - The number of atoms in the atomistic system

N_sites - The number of sites in the coarse-grained system

AA_pressures - The location of the xvg file containing the atomistic pressures

CG_pressures - The location of the xvg file containing the coarse-grained pressures

AA_volumes - The location of the xvg file containing the atomistic volumes

CG_volumes - The location of the xvg file containing the coarse-grained volumes

log_file - The location of the text file that will be created or overwritten to contain the output log

psi_output - The location of the psi output file that will be created or overwritten to contain the parametrization of the pressure correction (the result of the calculation)

Q_output - The location of the Q output file that will be created or overwritten to contain the correlation matrix from the calculation

g_cnt_output - The location of the g_cnt output file that contains the number of samples per bin for the bins used in the solution

7.3 Example config.ini file

An example config.ini file is provided below for an iterative run of a small heptane/toluene mixture system:

```
#Calculation settings
N_atoms: 4455
N_sites: 891
N_pres_coeff: 2
```



```

Pressure_units: bar
Volume_units: nm3
Energy_units: kj_per_mol
Basis_type: das_andersen
vmax: 118
vmin: 110
dv: 0.1
Iterative: 1
frac_cutoff: 0.1
Error_estimate: 1
Use_ref_Fv: 1

#Input Files
AA_pressures: aa_pressures.xvg
CG_pressures: press.dat
AA_volumes: aa_volumes.xvg
CG_volumes: vol.dat

ref_Fv_file: ../iter0/psi.dat

#Output Files
pres_decompose: decompose.xvg
log_file: log.txt
psi_output: psi.dat
Q_output: Q.dat
CG_Q_output: CG_Q.dat
AA_Q_output: AA_Q.dat
g_cnt_output: g_cnt.dat
AA_g_cnt_output: AA_g_cnt.dat
CG_g_cnt_output: CG_g_cnt.dat

```

7.4 Output file: psi

basis_type: das_andersen

This filetype is used for both output of a das_andersen basis calculation, as well as the input to an iterative das_andersen calculation as the ref_Fv file. There are three headings that are expected in this file in any order:

[**AVG_AA_VOLUME**] - Followed by a number on the next line which will be read in as the double-precision average volume of the atomistic system

[**PSI_COEFF**] - Followed by an integer on the same line denoting how many of the next lines to read in as double-precision psi coefficients

[N_SITES] - Followed by an integer on the next line which will be read in as the number of sites in the CG system

An example of the contents of this type of file follows:

```
[AVG_AA_VOLUME]
44.367584
[PHI_COEFF] 2
-43.993848
-330.013730
[N_SITES]
250
```

basis_type: (linear or delta) This filetype is used for input and output of tabulated basis functions. The output is in two columns separated by a comma, where the first column is the volume and the second column is the pressure correction (and value of psi) at that volume.

7.5 Output file: Q

The correlation matrix Q is printed out as in a text file with n_basis columns and n_basis rows, with the row and column of each entry corresponding directly to the row and column of that entry in Q.

References

8 Modified LAMMPS

We separately distribute our modified LAMMPS package from BOCS. The PKG-BOCS (formerly USER-BOCS) package is included in the official LAMMPS distribution when installing LAMMPS from the lammps/lammps repository.

The PKG-BOCS extension now has two separate functions:

- simulating the configuration independent volume potential, via LAMMPS's `fix bocs` command [Pressure Matched Models]
- simulating configuration dependent local density and square gradient potentials, via LAMMPS's `pair_style ldd` command [Local Density (LD) Potentials].

Up to date documentation and example input scripts are distributed in the official LAMMPS distribution, available from (linked here) the LAMMPS GitHub page and in future distributions from LAMMPS's official website. The following sections provide a brief overview for the capabilities contributed.

8.1 Pressure Matched Models

Pressure matched models are simulated in LAMMPS via the PKG-BOCS extension and `fix bocs`.

There are two input modes for F_V that are accepted. For both modes, the units of the input settings must be consistent with the units selected for your LAMMPS simulation.

8.1.1 Analytic F_V

This input mode for F_V is represented as a set of N coefficients for a Das & Andersen basis function. The full line for the bocs fix would look something like this:

```
#      id grp type {temperature params}    {pressure params}
fix    1  all bocs temp 303.0 303.0 100.0 cgoto 0.986 0.986 1000.0 \
      analytic 113606.3 1602 2 119273.87 -80079.04
#      type      v_avg      N      Nd psi1      psi2
```

The line break is only added for clarity within this document. The first half of the command mimics a standard `fix npt` command, with two notable changes: the type is now `bocs` instead of `npt` and the pressure type is `cgiso` instead of `iso`. The second half of the command details the extra parameters we must specify. The `analytic` keyword specifies the Das & Andersen basis function for F_V and determines the other arguments that are required. The arguments that follow are the average volume of the reference AA system, the number of sites in this CG system, the number of terms in the Das & Andersen basis function to use, and a list of the Das & Andersen psi coefficients.

8.1.2 Tabulated F_V

The pressure correction can also be input in a tabulated form read in from a file. The `fix bocs` line for this type of F_V input is also modified by additional terms that follow the default input:

```
fix          1 all bocs temp 303.0 303.0 100.0 cgoto 0.986 0.986 1000.0 \
      cubic_spline  delta_Fv.dat
#          interp_type  file

fix          1 all bocs temp 303.0 303.0 100.0 cgoto 0.986 0.986 1000.0 \
      linear_spline  delta_Fv.dat
#          interp_type  file
```

Here, we see two examples of the additional input required for a tabulated F_V : the interpolation type, and the file location. The specific choice of interpolation type determines how the program interpolates for sampled volumes between the supplied grid points. An example `delta_Fv.dat` is provided below for a small range of volumes. You should make sure that the `delta_Fv.dat` file you use spans well more than the range of volumes you plan to sample - the simulation will end with an error if the system samples a volume outside of the file's range.

```

# delta_Fv.dat
# vol  press    (make sure the units are correct)
50, -3064.57628
150, -3064.45344
250, -3064.33061
350, -3064.20778
450, -3064.08495
550, -3063.96212
650, -3063.83928
750, -3063.71645
850, -3063.59362
950, -3063.47079
1050, -3063.34796
1150, -3063.22512
1250, -3063.10229
1350, -3062.97946
1450, -3062.85663
1550, -3062.7338
1650, -3062.61096
1750, -3062.48813
1850, -3062.3653
1950, -3062.24247
2050, -3062.11964
2150, -3061.9968
2250, -3061.87397
2350, -3061.75114
2450, -3061.62831
2550, -3061.50548
2650, -3061.38264
2750, -3061.25981
2850, -3061.13698
2950, -3061.01415

```

8.1.3 Thermo_modify

```

#                press <name>
thermo_modify    press <FIX_ID>_press

```

This command tells LAMMPS to print the modified pressure (the pressure including the pressure correction) instead of the unmodified pressure (the pressure excluding the pressure correction). Replace “<FIX_ID>” with the ID you gave the fix_bocs command - e.g. above we used “1” for the ID, so we would use “thermo_modify press 1_press”.

8.2 Local Density (LD) potentials

Local Density potentials are enabled in LAMMPS via the `pair_style ldd` command. We have contributed How-to Guides, Pair-style documentation and example input scripts within our LAMMPS contribution. (See `lammps/doc`, `lammps/examples/PACKAGES/bocs` within the lammps distributed code.) We also host an “ear-marked” copy of this code in the noid-group repository (`lammps-BOCS2026:BOCSLD`) which likewise contains its own doc and examples for reproducing results published in the version 5 release paper (Lesniewski, DeLyser, Noid (2026)). [See <https://github.com/noid-group/lammps-BOCS2026/tree/BOCS-2026LD> for the “ear-marked” version and associated documentation] See the `HowtoLdd` page within this distribution for up to date directions on getting started. The below summarizes formatting instructions for properly outputting BOCS-compatible LD trajectories from the LAMMPS distribution.

8.2.1 Processing LD trajectory info in/with LAMMPS

The current implementation of this pair style enables users to write LAMMPS trajectories containing per-particle local densities, gradients of local densities, and associated interaction energies. BOCS can process custom dumped (See LAMMPS’s `dump custom` doc) LAMMPS trajectories in the format:

```
## LAMMPS Input File
dump my_dumpID all custom bocs_traj.lmp id type x y z vx vy vz fx fy fz
  dump my_dumpID all custom bocs_traj.lmp id type mol x y z vx vy vz fx fy fz
## Atoms trajectory section in BOCS readable LAMMPS trajectory file.
ITEM: ATOMS id type x y z vx vy vz fx fy fz
ITEM: ATOMS id type mol x y z vx vy vz fx fy fz
```

Trajectories formatted as such are recognized by BOCS using the “.lmp” file extension, and can be used with the `ldcalc` tool in order to post process local density / local density gradient information.

The `dump custom` format presented above can also be extended to contain local density and gradient of local density information if one would like to save the calculated local density / local density gradient information from LAMMPS directly.

Printing local density and local density gradient information can be accomplished using the `fix pair` command documented on LAMMPS’s website. Table 6 contains all of the `fix pair` keywords specific to `pair_style ldd`. These fixes can be dumped with LAMMPS’s `dump custom` command. The “HowtoLDD” page within the LAMMPS documentation provides more detailed instructions for using `fix pair` to print local density and local density gradient information, the `lammps/PACKAGES/examples/bocs` folder also contains an example script writing this information to a `dump custom` command. e.g.

```
dump my_dumpID all custom bocs_traj.lmp id x y z vx vy vz \
```

```
fx fy fz fix_IDA[*] fix_IDB[*]
```

```
## Atoms trajectory section (not yet BOCS readable)
ITEM: ATOMS id type mol x y z vx vy vz fx fy fz \
fix_IDA[1] fix_IDA[2] fix_IDB[1] fix_IDB[2]
```

wherein `fix_ID` is the name of the `fix pair` instance the user employs and the linebreak has been placed for readability. (As in prior examples, the line break is added only for clarity in this document.)

BOCS can read local density information printed from LAMMPS this way, but it does so naively, by trying to find headers labeled via a certain keyword in a certain order. The fix headers exemplified above can be re-labeled in LAMMPS using the `dump_modify colname` command. We record the order to print each local density data set along with the keyword BOCS expects to read labeling it in table 6.

fix pair key-word	Description	BOCS trajectory header label
<code>local_density</code>	$\rho_{\tau I}$ (the partial local density) for each atom I and each type $\tau \in \{1...N_t\}$	<code>lddensτ</code>
<code>energy</code>	$U_{\tau \tau_I}(\rho_{\tau I})$ (the energy contribution from each partial local density) for each atom I and each type $\tau \in \{1...N_t\}$	<code>lddnrgτ</code>
<code>grad_density</code>	$\nabla_I \rho_{\tau I}$ (the gradient of the partial local density) for each atom I and each type $\tau \in \{1...N_t\}$	<code>gradxτ gradyτ gradzτ</code>
<code>grad_energy</code>	$U_{\nabla;\tau \tau_I}(\rho_{\tau I}) \nabla_I \rho_{\tau I} ^2$ (the SG potential contribution) for each atom I and each type $\tau \in \{1...N_t\}$	<code>gradnrgτ</code>
<code>total_energy</code>	The total potential energy coming from LD/SG interactions for particle I	<code>ttltnrg</code>

Table 6: By default LAMMPS will label columns of custom trajectory computes and fixes via the name of the computeID or fixID the user put into LAMMPS. BOCS reads trajectories naively, by trying to find headers labeled via a certain keyword in a certain order. The order is: local density and local density energy of each type, then gradient density and gradient energies of each type, then total energy. The last column of this table lists the rename pattern header that BOCS looks for.

Within each LD/SG info type, data is sorted by surrounding atom type. For example, a simulation that employs two distinct local density types $\tau \in 1, 2$ can be analyzed with BOCS directly if the custom dump ATOMS header is formatted as:

```
ITEM: ATOMS id type x y z vx vy vz fx fy fz \  
lddens1 ldnrg1 lddens2 ldnrg2 \  
gradx1 grady1 gradz1 gradnrg1 \  
gradx2 grady2 gradz2 gradnrg2 lddttlrg
```

where we have placed the line breaks for readability in this document.

9 Tutorials

9.1 Lennard-Jones Recovery Tutorial

In this first tutorial we will demonstrate how to use the BOCS toolkit for one of the simplest possible cases: the recovery of the Lennard-Jones nonbonded pair interaction. No actual coarse-graining takes place for this process. Rather, the cgff tool is used to calculate the nonbonded force interaction given a simulated trajectory of Lennard-Jones particles.

This and all following tutorials assume that you are using the default name for all GROMACS and BOCS executables, and that you have them in your PATH variable.

To minimize the size of files stored in the BOCS toolkit distribution, we have not bundled any actual trajectory files. Rather, we provide the input files so that you can generate the trajectories for yourself. For this tutorial, the simulation is very quick.

Input files for this tutorial can be found in the `tutorials/lj_recovery` folder.

9.1.1 AA Simulation

Files:

- LJ.mdp
- LJ.gro
- LJ.top

Providing detailed instructions on the use of GROMACS is beyond the scope of this tutorial. Refer to the documentation provided at <http://manual.gromacs.org/documentation/> for the version of GROMACS you are running. This tutorial was developed using GROMACS 5.1.4.

Briefly, this simulation consists of 729 Lennard-Jones particles at constant pressure and temperature. This simulation should run in less than 10 minutes on a single processor. Run the simulation with the following commands:

```
$ gmx_mpi grompp -c LJ.gro -p LJ.top -f LJ.mdp -o LJ.tpr
$ gmx_mpi mdrun -s LJ.tpr -o LJ.trr -e LJ.edr
```

9.1.2 Force Matching

Files:

- LJ.trr
- inp.txt
- par.txt
- LJ.btp

Before force matching, you need a .btp file corresponding to the .trr file. First, dump the .trr file to a text file:

```
$ gmx_mpi dump -s LJ.tpr > LJtpr.dump
```

Second, translate the text file to a .btp file

```
$ translator -f LJtpr.dump o LJ.btp
```

Now that we have generated the trajectory and a BOCS topology file, we can proceed with the force matching calculation. This calculation requires two input files, par.txt and inp.txt. The par.txt file specifies the type of force matching calculation we are performing. The inp.txt file indicates which trajectory file we are using as input. Refer to the user's manual for a detailed description of each of these files.

Briefly, we are force matching a single pair interaction named LJLJ, between LJ particles in the Lennard-Jones trajectory we just generated. This interaction is being fit with a linear basis with a grid spacing of 0.005 nm, from 0.2 nm to 1.2 nm.

You can run the force matching calculation using the following command. Again, this system is small and simple, so the calculation will proceed very quickly on a single processor.

```
$ cgff
```

As indicated in the user manual, running the cgff tool generates a set of output files. For your reference, a set of this output files has been included in the `tutorials/lj_recovery/results` folder. The most immediately interesting of these files is `f_forces.Pair_interaction.total.LJLJ.dat`, which contains the force matching result of the LJLJ interaction.

The full set of results files generated includes:

- `b_forces.Pair_Interaction.total.LJLJ.dat` - the projection of the force field, `b`, as calculated from the forces
- `f_forces.Pair_Interaction.total.LJLJ.dat` - the calculated force matched force
- `g_cnt.Pair_Interaction.total.LJLJ.dat` - a normalized histogram of sampling along the pair interaction
- `g.Pair_Interaction.total.LJLJ.dat` - the metric tensor matrix `G`
- `L_coeff.Pair_Interaction.total.LJLJ.dat` - the Laplacian contribution that incorporates the appropriate Jacobian factor

A set of save state files, `save*.dat`, are also generated by this calculation. These filenames correspond to internal variables in the cgff tool, and are printed right before solving the set of linear equations to determine the force-matched force field. They can be used as input into a second half calculation for re-calculating the force field without looping over the trajectory again (see user's manual).

An additional check, which demonstrates that force-matching can accurately recover a given potential is to plot the LJ potential that was used in the simulation and compare it to the resulting potential from force-matching. The sigma and epsilon values are given in the LJ.top file, and these can be used to generate an LJ potential function.

9.2 1-Site Methanol Tutorial

In this tutorial, we will perform some simple coarse-graining and calculate a force field for 1-site methanol. This model maps each methanol molecule to its center of mass, and has a single interaction type: the nonbonded pair interaction between methanol sites. Despite also having a single site type in the coarse-grained representation, this calculation is more computationally demanding than the Lennard-Jones recovery tutorial. This is because this tutorial is actually performing a coarse-graining operation, and thus needs a larger amount of sampling. Additionally, the CG methanol pair interaction has a longer range, so a larger box must be simulated to accommodate its range. This simulation therefore contains 968 methanol molecules, run for 10 ns. Files for this tutorial can be found in the `tutorials/1-site_methanol` folder.

9.2.1 AA Simulation

Files:

- `methanol.gro`
- `methanol.mdp`
- `methanol.top`
- `methanol_oplsaa.itp`

You will want to run this simulation on HPC resources if at all possible, as generating a smooth CG force curve for this system requires significant simulation. We recommend running the AA simulation step overnight on 20 cores. On 20 cores, this simulation should get approximately 50 ns/day, however variability is to be expected between different HPC resources. You can set up and run the simulation with the following commands:

```
$ grompp -f methanol.mdp -c methanol.gro -p methanol.top -o methanol.tpr
```

```
$ mpirun -np 20 mdrun -s methanol.tpr -o methanol.trr -e methanol.edr
```

9.2.2 Mapping

Files:

- `methanol.trr`
- `methanol.tpr`
- `map.methanol.top`

Now that we have generated a set of atomistic configurations, we must map them to the corresponding coarse-grained representation. The file `map.methanol.top` contains the information necessary for mapping from the AA to the CG representation. The anatomy of a mapping topology file is discussed in more detail in the user's manual. Briefly, the AA representation contains 6 atoms, and this mapping file specifies that the CG site for each methanol molecule shall be placed at the center of mass of the molecule, for all 968 methanol molecules in the system.

The first step we must take is to make all of the molecules whole across periodic boundary conditions. The `cgmap` and `cgff` tools are not very clever with respect to periodic boundaries, so we must translate for them using the command:

```
$ trjconv -f methanol.trr -force -pbc whole -s methanol.tpr
-o whole_methanol.trr
```

The BOCS toolkit uses the `cgmap` tool to transform configurations from the AA to the CG representations. The `cgmap` tool can produce the mapped trajectory (`.trr`) file as well as a mapped structure file (`.gro`), corresponding to the last frame in the trajectory. This `.gro` file will be used later to generate a BOCS `.btp` file as well as input for any CG simulations run. You can generate this mapped `.trr` and `.gro` files using the `cgmap` command:

```
$ cgmap -f whole_methanol.trr -p map.methanol.top -c cg_methanol.gro -o cg_methanol.trr
```

Mapping is a computationally simple operation, so this command takes only a minute or two to complete on the command line. If you would like to run it through a scheduler on HPC resources, only use 1 processor as `cgmap` is a serial executable.

9.2.3 Force matching

Files:

- `inp.txt`
- `par.txt`
- `cg_methanol.trr`
- `cg_methanol.btp`

Now, finally, we can perform some actual force matching. Up until this point the only tools which have required topology information have been Gromacs tools, and thus we have used `.tpr` files. The `cgff` tool requires a BOCS topology file (`.btp`) so we must create one. The simplest way to do this is to first create a coarse-grained `.tpr` file, then use the BOCS translator to translate it to a `.btp` file. To do this, Gromacs requires three input files to `grompp` into a `.tpr` file:

1. The `cg_methanol.top` file is a normal Gromacs `.top` file for a system of 1-site molecules, except that all interaction functions must be set to 1. This is important later for the `cgff` tool to determine which interactions are present in your system. In general, this file must be made manually, however in this tutorial we provide you with it.
2. The `cg_methanol.mdp` must be a valid `.mdp` file, but most of its contents do not matter, as we will not be using the resulting `.tpr` file for any simulations, however, be sure that you are not constraining any bonds.
3. The `cg_methanol.gro` file was generated in the previous `cgmap` step, and should be used here.

Once the CG `.tpr` file is generated, it must be dumped to a text file, which then must be translated to a `.btp` format. This can be done using the following commands:

```
$ grompp -f cg_methanol.mdp -p cg_methanol.top -c cg_methanol.gro
-o cg_methanol.tpr
```

```
$ gmx_mpi dump -s cg_methanol.tpr > cg_methanol_tpr.dump
```

```
$ translator -f cg_methanol_tpr.dump -o cg_methanol.btp
```

As before, the `inp.txt` and `par.txt` files (included in the `tutorials/1-site_methanol/force_matching` folder) specify the trajectory file and the type of force matching calculation we are performing. This `par.txt` file is similar to that for the Lennard-Jones recovery tutorial because both systems are 1-site per molecule in the CG model. This is another case where you will want to run the calculation on approximately 20 cores using HPC resources, however, this command should finish in a couple minutes. The command to run the calculation is:

```
$ mpirun -np 20 cgff
```

This will generate an analogous set of files to those seen in the Lennard-Jones recovery tutorial.

9.2.4 Test Simulation

- `f_forces.Pair_Interaction.total.MEOMEO.dat`
- `cg_methanol.top`
- `cg.methanol.gro`
- `cg_methanol.mdp`
- `cg_methanol.ndx`

Now that we have calculated the CG pair force via force matching, let's use it in a simulation. First, we must refine the output of the `cgff` tool for use in a simulation. This refining is done by the `tables` tool, and takes the form of extending the force back to zero, and out to the cutoff required by GROMACS, as well as changing to the required `xvg` file format.

```
$ tables f_forces.Pair_Interaction.total.MEOMEO.dat table_MEOMEO.xvg
  nb Bspline 3.50
```

The `tables` tool will error if there are any unsampled bins in `f_forces.Pair_Interaction.total.MEOMEO.dat`. To address these, you may interpolate between points using a method of your choosing, or simply delete the bins up to the missing bin. Since unsampled bins typically occur at very short distances, we recommend simply deleting the bins up to the missing bin.

We must also prepare a `.tpr` file for this simulation. Note that the `mdp` file specifies the MEO energy group, and the MEO MEO energy group table. This causes GROMACS to look for our `table_MEOMEO.xvg` file to define the MEO MEO pair interaction. Also note that the setup of the pair interaction in the `cg_methanol.top` file uses function type 1, with coefficients of 0 and 1. This identifies the column in our table file that will be read for the pair interaction. The index file is also necessary for this table file interaction scheme, as it identifies which sites have the MEO type to the `edr` file. In this case, it is all of them. A Gromacs index (`.ndx`) file is required here due to the use of tabulated potentials, and we have provided one here. Generally, this file can be made using the Gromacs `make_ndx` tool.

```
$ grompp -f cg_methanol.mdp -p cg_methanol.top -c cg_methanol.gro
  -n cg_methanol.ndx -o cg_methanol.tpr
```

Finally, you can run the simulation. On 8 processors, this simulation takes a little more than an hour.

```
$ mpirun -np 8 mdrun -s cg_methanol.tpr -o cg_methanol.trr
  -e cg_methanol.edr
```

You can compare the results of this CG simulation to those of the mapped AA simulation. A common property to compare between the two would be the methanol-methanol radial distribution function, which can be accessed with the Gromacs `rdf` tool. We have also provided sample `rdfs` to compare to.

9.3 Extended Ensemble and PMatching Tutorial: Heptol Mixtures

This tutorial will take you through the process of force matching an from an extended ensemble of systems, where we will derive a force field that is optimal across a set of related

chemical systems (the extended ensemble). For this example, we will perform this calculation for a set of heptane:toluene mixtures of differing compositions, and obtain a single force field that can be used to simulate systems with any ratio of heptane:toluene. The CG representations used here are 3-site models for both molecules.

9.3.1 AA Simulations

Files:

- X_Y_HEP_TOL.gro
- X_Y_HEP_TOL.top
- X_Y_HEP_TOL.mdp
- heptane.itp
- toluene.itp

In order to perform this extended ensemble force matching calculation, we must first generate the extended ensemble of atomistic simulations for use as input. This tutorial provides input files for atomistic simulations of 5 heptane:toluene systems in the folder: `tutorials/heptol_xn/aa_simulations`

The gro files contain configurations from already equilibrated systems, so you need only `grompp` and `mdrun` the files. These systems contain approximately 10,000 atoms, so they will parallelize efficiently onto around 50 cores. These simulations need around 50 ns of simulation time to get enough sampling, and there are 5 such systems, so you will likely want to run them as a job on HPC resources.

The commands you will use are as follows (substituting the composition #'s for X and Y):

```
$ grompp -f X_Y_HEP_TOL.mdp -c X_Y_HEP_TOL.gro -p X_Y_HEP_TOL.top  
-o X_Y_HEP_TOL.tpr
```

```
$ mpirun mdrun -s X_Y_HEP_TOL.tpr -o X_Y_HEP_TOL.trr -e X_Y_HEP_TOL.edr
```

9.3.2 Mapping

Files:

- X_Y_HEP_TOL.trr
- X_Y_HEP_TOL.tpr
- map.X_Y_HEP_TOL.top
- cg.X_Y_HEP_TOL.top

- `cg.X_Y_HEP_TOL.mdp`
- `cg.X_Y_HEP_TOL.ndx`

As before, you will need to make the trajectories whole with respect to their periodic boundary conditions.

```
$ trjconv -f X_Y_HEP_TOL.trr -force -pbc whole -s X_Y_HEP_TOL.tpr
-o whole_X_Y_HEP_TOL.trr
```

You will again use the ‘cgmap’ tool to generate a CG gro file, and to map each of the 5 atomistic simulations to their CG representations. You can generate this gro file using the cgmap command:

```
$ cgmap -f whole_X_Y_HEP_TOL.trr -p map.X_Y_HEP_TOL.top -c cg.X_Y_HEP_TOL.gro
```

And map the trajectories with these commands:

```
$ grompp -f cg.X_Y_HEP_TOL.mdp -p cg.X_Y_HEP_TOL.top -n cg.X_Y_HEP_TOL.ndx
-c cg.X_Y_HEP_TOL.gro -o cg.X_Y_HEP_TOL.tpr
```

```
$ gmxdump -s cg.X_Y_HEP_TOL.tpr > cg.X_Y_HEP_TOL_TPR.dump
```

```
$ translator -ft top -fi cg.X_Y_HEP_TOL_TPR.dump -ipt dump -fo cg.X_Y_HEP_TOL.btp
-opt bocs
```

```
$ cgmap -f whole_X_Y_HEP_TOL.trr -ipt trr -p map.X_Y_HEP_TOL.top -s cg.X_Y_HEP_TOL.btp
-st bocs -o cg.X_Y_HEP_TOL.trr -opt trr
```

9.3.3 Force Matching

Files:

- `inp.txt`
- `par.txt`
- `cg.1_4_HEP_TOL.trr`
- `cg.2_3_HEP_TOL.trr`
- `cg.1_1_HEP_TOL.trr`
- `cg.3_2_HEP_TOL.trr`
- `cg.4_1_HEP_TOL.trr`

- corresponding .btp files

The `inp.txt` and `par.txt` files for the extended ensemble calculation can be found in the folder:

`tutorials/heptol_xn/coarse_graining/extended_ensemble`

Note that both files point to a set of .tpr and .trr files corresponding to those used in the previous mapping step. If you moved these files outside of their respective folders or named them differently, you will need to update the paths in `inp.txt` and `par.txt` to reflect this.

The `par.txt` file specifies the interactions that will be force matched in this calculation. In this case, all of these interactions are present in each of our source systems, as they all contain mixtures of heptane and toluene. It is also possible to include interactions here that are present in only some of the extended ensemble systems.

You can run this calculation as before, in the same folder as `inp.txt` and `par.txt`, with:

```
$ mpirun cgff
```

Since these are larger systems and there are 5 of them that the calculation needs to loop through, this calculation is best submitted as a job on an HPC system. It parallelizes very well up to a very large number of processors, so the more processors you use the better. In our tests, it takes around 48 hours on 64 processors.

This will generate a large set of output files, which are analogous to those generated in the methanol tutorial. Each of the per-interaction files will be labeled with the interaction name specified in `par.txt`. We will work directly with the `f_force.XXX.total.YYY.dat` force field files in the coming steps.

We have also included `inp.txt` and `par.txt` files for generating independent force fields for each of the individual systems, if you would like to do so for comparison purposes. These files are found in the corresponding folders:

`tutorials/heptol_xn/coarse_graining/x_y_heptol`

9.3.4 Make Table Files

Files:

- `f_forces.Pair_Interaction.total.CMCB.dat`
- `f_forces.Pair_Interaction.total.CMCF.dat`
- `f_forces.Pair_Interaction.total.CMCM.dat`
- `f_forces.Pair_Interaction.total.CTCB.dat`
- `f_forces.Pair_Interaction.total.CTCF.dat`
- `f_forces.Pair_Interaction.total.CTCM.dat`
- `f_forces.Pair_Interaction.total.CTCT.dat`

- `f_forces.Pair_Interaction.total.CBCB.dat`
- `f_forces.Pair_Interaction.total.CFCB.dat`
- `f_forces.Pair_Interaction.total.CFCF.dat`
- `f_forces.Angle.total.Angle_CT-CM-CT.dat`
- `f_forces.BondStretch.total.Bond_CB-CB.dat`
- `f_forces.BondStretch.total.Bond_CF-CB.dat`
- `f_forces.BondStretch.total.Bond_CT-CM.dat`
- `make_gmx_table.sh`

As is sometimes the case for force matching calculations, there are several forces that need to be processed and converted into table files. Since we are going to continue on to pressure-match these systems as well, we will need to generate table files for both LAMMPS and GROMACS.

We will start with processing the files for use with GROMACS. The script `make_gmx_table.sh` contains the list of commands needed to perform this processing for the full set of table files. You can run it via:

```
$ sh make_gmx_table.sh
```

This should generate the full set of tables in the folder:
`/tutorials/heptol_xn/coarse_graining/table/`

Now, we can convert these table files for use in LAMMPS. This conversion makes use of a script in the `YYY_toolkit/scripts` directory called `translate_table.py`. It is a simple python script that converts table files from GROMACS format and units to LAMMPS ‘real’ units and format. You should have this script on your path. If so, then you can move into the ‘tables’ folder and run the command:

```
$ sh ../lammmps_table.sh
```

This is a helper script to run the full set of table conversion commands. Inspect this script to see the form of the `translate_table.py` commands; it is also documented within the python script itself.

9.3.5 Pressure Matching: Rerun

Files:

- `cg.1_1_HEP_TOL.trr`
- `cg.rerun.1_1_HEP_TOL.gro`

- `cg.rerun.1_1_HEP_TOL.mdp`
- `cg.rerun.1_1_HEP_TOL.top`
- `cg.rerun.1_1_HEP_TOL.ndx`

At this point, we could go ahead and run a CG simulation of any of our heptane:toluene mixtures. However, if we do so under constant pressure (NPT) conditions, the simulations will blow up. This is because the pressure of the CG simulation is substantially higher than that of the underlying AA system - a common problem with bottom-up coarse-graining methods.

Therefore, we will instead proceed by determining a volume-dependent pressure correction for use with the CG simulation. Here, we will generate a pressure correction for the 1:1 heptane:toluene system. You can follow an analogous procedure for any of the other compositions, as the pressure matching calculations for extended ensemble calculations are performed independently of the other systems in the ensemble.

The first step in this process is to determine an approximate correction based on the pressure of the mapped trajectory with the CG force field. We will do so using the ‘mdrun-rerun’ command in GROMACS, which allows us to evaluate the forces (and therefore the pressures) over an existing trajectory with a new force field.

Note that the input files (mdp and top in particular) are of the type used for CG simulation, not for mapping. This means that the .mdp file has a fully filled out ‘energygrps’ entry for all pair interactions in the system, and that the .top file has all intramolecular interactions set to function type 8.

Also note that as of GROMACS 4.5.3, there is a bug with rerunning trajectories when you compile with openMPI that causes nonsense energy outputs. Therefore, we recommend that you perform the rerun with an mdrun executable compiled without openMPI.

Rerun the trajectory from the `tutorials/heptol_xn/pressure_matching/rerun` folder using the following commands:

```
$ cp ../../coarse_graining/extended_ensemble/table/*xvg ./

$ grompp -c cg.rerun.1_1_HEP_TOL.gro -p cg.rerun.1_1_HEP_TOL.top
  -f cg.rerun.1_1_HEP_TOL.mdp -n cg.rerun.1_1_HEP_TOL.ndx
  -o cg.rerun.1_1_HEP_TOL.tpr

$ mdrun-noMPI -rerun ../../coarse_graining/1_1_heptol/cg.1_1_HEP_TOL.trr
  -s cg.rerun.1_1_HEP_TOL.tpr -e rerun.edr -nt 8
```

This rerun should proceed relatively quickly, as there are 1000x fewer frames for which to compute forces compared to the original atomistic simulation, and we are calculating the forces in the CG system, which has fewer particles and therefore fewer pair interactions. Once the rerun is complete, extract the CG pressures and volumes using the GROMACS `g_energy` utility:

```
$ g_energy -f rerun.edr -o cg_volumes.xvg
```

```
$ g_energy -f rerun.edt -o cg_pressures.xvg
```

While you're at it, extract the pressures and volumes from the corresponding atomistic simulation as well:

```
$ g_energy -f ../../aa_simulations/1_1_heptol/1_1_HEP_TOL.edr -o aa_volumes.xvg
```

```
$ g_energy -f ../../aa_simulations/1_1_heptol/1_1_HEP_TOL.edr -o aa_pressures.xvg
```

9.3.6 Pressure Matching: Zeroth Iteration

Files:

- aa_volumes.xvg
- aa_pressures.xvg
- cg_volumes.xvg
- cg_pressures.xvg
- config.ini
- cgheptol.lmp
- data.cgheptol

Copy the xvg files you just generated into `tutorials/heptol_xn/pressure_matching/iteration_zero`, and use the `p_match` tool to generate the approximate pressure correction with the command: `p_match config.ini`

The `config.ini` file contains all the settings for the pressure matching calculation. Here, we are using an analytic basis of the form recommended by Das and Andersen with two terms. This is equivalent to directly correcting the pressure and the compressibility of the CG model. A number of diagnostic files will be generated when running this command, but we are most interested in `psi.dat`, which contains the two coefficients for our pressure correction. Our `psi.dat` file contains the following:

```
[AVG_AA_VOLUME]
113.610979
[PSI_COEFF] 2
101.348739
14.024344
[N_SITES]
1602
```

Your average volume and coefficients should be similar but not identical, as our calculations will differ by the particular AA simulations used as input.

The psi coefficients are given in units of $\text{bar} \cdot \text{nm}^3$, and the average volume is given in nm^3 . In order to use these with LAMMPS, we must convert them to the corresponding LAMMPS 'real' units, which use atm for pressures and $\text{\AA}^3$ for volumes. These conversions are:

$$X \text{ nm}^3 = \frac{1000 \text{ \AA}^3}{1 \text{ nm}^3} = 1000 \text{ \AA}^3 \quad (3)$$

$$X \text{ bar nm}^3 * \frac{1 \text{ atm}}{1.01325 \text{ bar}} * \frac{1000 \text{ \AA}^3}{1 \text{ nm}^3} = 986.923 \text{ atm \AA}^3 \quad (4)$$

Edit these converted coefficients and the average volume into the `cgheptol.lmp` file in the `test_sim` folder, replacing the angular brace placeholders on line 70 of this file, making sure that there are not remaining braces in the final line.

Note that the `data.cgheptol` file contains the coordinates and specifies the connectivity of the molecules in this system. It was generated using the (unsupported) script `YYY_toolkit/scripts/translate_top.py` to pull information from `.gro`, `.top`, and `.ndx` files.

Assuming you have built LAMMPS for use with openMPI, you can run the simulation via:

```
$ mpirun lmp_linux < cgheptol.lmp &> out.dat
```

This will generate a trajectory file, `outfile.xtc`, and a set of `.dat` files that print various thermodynamic quantities. We are most interested in `vol.dat` and `press.dat`, which contain the volume and pressure (converted to GROMACS units). We print in GROMACS units so that we can directly use the output of this simulation in the next step of the pressure matching process, which will compare the PV curve estimated by rerunning over the mapped ensemble with this CG simulation.

Once this simulation is done, plot the distribution of volumes in the AA simulation against the distribution of volumes in this CG simulation using your favorite plotting tool. You will see that the pressure correction used here is not quite right, and the average density and compressibility of the CG simulation are therefore different than those of the AA model. This is because of subtle structural differences in the simulated CG ensemble compared to the mapped ensemble.

9.3.7 Pressure Matching: First Iteration

Files:

- `aa_volumes.svg`
- `aa_pressures.svg`

- `vol.dat`
- `press.dat`
- `config.ini`
- `cgheptol.lmp`
- `data.cgheptol`

To correct for these structural differences, we can apply an iterative method to update the pressure correction for this CG model. To do so, we will compare the pressure-volume (PV) curve of the CG simulation in the last step to the PV curve of the AA simulation, and infer an update to the pressure correction based on this difference.

To perform this calculation, copy the AA pressure and volume files into the folder: `tutorials/heptol_xn/pressure_matching/iteration_one` as well as the `press.dat` and `vol.dat` files generated during the previous iteration. You can then run the command:

```
$ p_match config.ini
```

To generate an updated set of psi coefficients based on the difference between the simulated AA and CG PV curves. This calculation will read in the previous psi coefficients from the `psi.dat` file in the `iteration_zero` folder, and generate a new `psi.dat` file here that incorporates the updates.

You can then proceed to simulate with these coefficients in the same way as before - edit in the average volume and converted coefficients into `cgheptol.lmp`, then simulate with

```
$ mpirun lmp_linux < cgheptol.lmp &> out.dat
```

When you plot the distribution of volumes this time, it should be closer to reference AA distribution. You can continue iterating over this process by generating new sets of coefficients and simulating with them until the volume distribution is satisfactorily converged. For most systems, this takes fewer than 10 iterations.

9.4 Local Density Tutorial

In this tutorial, we will repeat the 1-site methanol tutorial from earlier, but with the addition of a local density basis set. As before, we will be mapping each methanol molecule to its center of mass. However this time, in addition to the pair potential built earlier, we will be adding a non-bonded interaction that is a function of the local density of sites surrounding a given CG methanol. Such potentials have been studied in several works in the Noid Lab, here we reference the first two, which are the papers that BOCS's LD software was originally written for. [Delyser, Noid JCP 2017](#) [Delyser, Noid JCP 2019](#) We will continue with the same system used in the 1-site methanol tutorial for this tutorial.

9.4.1 AA Simulation / Mapping

This step should already have been completed as part of the 1-site methanol tutorial

9.4.2 Calculating Local Densities

9.4.2a. Using the ldcalc tool

Files:

- `cg_methanol.trr` [mapped from 1-site tutorial trajectory]
- `cg_methanol.btp` [generated during 1-site tutorial forcematching]
- `LD_info.txt` [Supplied]

In order to use force-matching to parameterize a CG local density potential, we first need to compute and store LDs and LD gradient info. To do this, its a simple as running the `ldcalc` executable introduced in section 2 of this manual. Since `ldcalc` will analyze the whole trajectory, this executable should be submitted as a job to a cluster (don't run it on the command line). `ldcalc` is openMP parallelized, so it can be run on multiple threads. Here's an example script running `LDcalc` on multiple threads for a SLURM managed cluster:

```
# EXAMPLE.sh
#!/bin/bash
#SBATCH --job-name="YOUR_JOB_NAME"
#SBATCH --output="YOUR_OUTPUT_FILE.out"
#SBATCH --partition=YOUR_PARTITION_NAMED
#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=24
#SBATCH --mem=16G
#SBATCH --account=YOUR_ACCOUNT
#SBATCH --export=ALL
#SBATCH -t 03:00:00

ml bocs
# if you do not have bocs in a lua file, load the gcc, intel-mkl library,
# and openmpi libraries you used for compiling it

export OMP_NUM_THREADS=24
# Be sure to include this or you won't use all 24 cores requested

ldc="/path/to/ldcalc"
```

```

${ldc} -s cg_methanol.btp -f cg_methanol.trr -i LD_info.txt \
-o cg_methanol_traj_w_LDs.btj -self
# You can also do this one one line, the escape "\"" is to format for this manual

#if you have the lua files for loading bocs you can
#replace ${ldc} with ldcalc explicitly

# END EXAMPLE.sh

```

The arguments for the ldcalc command are explained below.

- `-s cg_methanol.btp` cg topology file. has to be a .btp file for the CG system.
- `-f cg_methanol.trr` cg trajectory file. this should be the .trr file came out of cgmap.
- `-o cg_methanol_traj_w_LDs.btj` new cg trajectory file. make sure it has the file extension .btj (bocs trajectory).
- `-i: LD_info.txt` file defining local densities to calculate, explained more below
- `-self` option to include a "self" contribution to the LD, explained more below

The contents of LDinfo.txt are explained on the following page.

9.4.2b. Understanding LD_info contents

This subsection contains information for understanding and tailoring the LD_info file. The purpose of the LD_info file is to define the local density boes will calculate, thus it is imperative to have a basic understanding of indicator functions to be able to tailor LD_info appropriately.

The local density (LD) of sites around a given site I, ρ_I , is defined as the sum of a normalized weighting function acting on the pair distances between I and the sites that surround it. e.g.

$$\rho_I = \sum_{J \in \mathcal{N}(I)} \bar{w}(r_{IJ}) \quad (5)$$

where:

$\rho_I \rightarrow$ LD of particles surrounding site J

$r_{IJ} \rightarrow$ Distance between a particle I and its neighbor J

$\mathcal{N}(I) \rightarrow$ The set of all particles within $r < r_{cut}$ [some neighbor list cutoff] of a particle I. When -self is specified above, this sum includes particle I as one of its own neighbors, this shifts the definition of the LD by $\bar{w}(0)$

$\bar{w}(r_{IJ}) \rightarrow$ The normalized weighting function, defined as $w(r)/(\int_0^{r_c} w(r) * 4\pi r^2 dr)$

The weighting function is a monotonically decreasing function of r that vanishes for distances greater than some cutoff. BOCS supports several weighting or “indicator” functions, for the purpose of this tutorial we will be using the lucy weighting function:

$$w(r) = (1 - r/r_c)^3(1 + 3r/r_c)\Theta(1 - r/r_c) \quad (6)$$

where:

r is the pair distance between particles

r_c is the weighting function cutoff

Θ denotes the Heaviside function.

The purpose of the LD_info file is to provide a definition of local density for each component of your simulated system. This includes the number of LD site types, the choice of indicator function, along with any choices for hyperparameters within the indicator function. e.g. the lucy function has one hyperparameter r_c , which we have selected for this tutorial to be 0.52685 nm.

```
#LD_info.txt
1                      ; no. site types for LD
0 MEO                 ; define site type 0
0 0 lucy 0.0 0.52685  ; define LD for site type 0 surrounded by 0
#END LD_info.txt
```

Generic “sections” of LD_info.txt files

- Section 1: the first line. simply an integer number that says how many site types you have in your simulation. we’ll call this “n”.

- Section 2: the next n lines. each line contains the index (starting from 0) and the site type name
- Section 3: the next $n*n$ lines. each line contains 3 integers followed by 2 floats.
 - The first column is the central site type.
 - The second column is the site type we’re looking for in the neighborhood of particles of the type given in the first column.
 - The third column is either “shell”, “sphere”, “smooth”, or “lucy” depending on the indicator function you want to use.
 - The fourth column is r_0 for the indicator function. If you use the lucy function, this must be 0.0. If you use any of the three smoothed heaviside function indicators, it’s whatever you want the start of the transition region to be.
 - The fifth column is r_c for the indicator function. It’s the end of the region over which LDs will be calculated. Both r_0 and r_c should be given in nanometers.

To further understand LD_info.txt type files, below is another example for a two site-type system.

Another LD_info Example

#LDinfo.txt Contents

```
2                      ; no. site types for LD
0 LJ1                  ; define site type 0
1 LJ2                  ; define site type 1
0 0 lucy 0.0 0.52685   ; define LD for site type 0 surrounded by 0
0 1 lucy 0.0 0.52685   ; define LD for site type 0 surrounded by 1
1 0 lucy 0.0 0.52685   ; define LD for site type 1 surrounded by 0
1 1 lucy 0.0 0.52685   ; define LD for site type 1 surrounded by 1
```

END Another LD_info Example

9.4.3 Force Matching

Files:

- `cg_methanol_traj_w_LDs.btj` [generated by `ldcalc` in previous section]
- `hilo_LDs.dat` [generated by `ldcalc` in previous section]
- `par.txt` [provided, needs to be edited in this section]
- `inp.txt` [provided, needs to be edited in this section]

- `cg_methanol.btp` [from 1-site tutorial]

Now that we have local density information in the trajectory we want to force-match, we can go ahead and generate our CG force functions using the `cgff` tool. As in prior tutorials, the parameters for the CG potential are specified in `par.txt`. As in any other interaction, we need to provide a range of order parameters that we want forces over. The `par.txt` provided is nearly complete, but filling in the local density range as well as any relative paths to `cg_methanol_traj_w_LDs.bty` and `cg_methanol.btp` has been left as an exercise for the user.

Specifically:

```
#excerpt from par.txt
[Site_Types] 1
MEO
[End Site_Types]

[Inter_Types] 2
!inter_name      inter_type    basis  dr  R_min R_max spline_order n_smooth
Pair_MEO_MEO     Pair_Interaction  Bspline  0.01 0.000 2.000 4  0
!inter_name      inter_type    basis  drho Rho_min Rho_max spline_order n_smooth |LD defs
LD_MEO_MEO       Local_Density  Bspline  0.1  LDLO LDHI 4  0 lucy 0.0 0.525265
[End Inter_Types]

[Pair_Interaction] 2
!inter_name type1 type2
Pair_MEO_MEO  MEO  MEO
LD_MEO_MEO   MEO  MEO
[End Pair_Interaction]

#END excerpt from par.txt
```

“LDLO” and “LDHI” should be provided by the user based on the reported low and high local densities inside of `hilo_LDs.dat`. We would suggest using the floored/rounded versions of the reported numbers so that LDLO and LDHI are rounded to the 0.1’s place and inclusive of all sampled values. Once the `inp.txt` and `par.txt` files are complete, forcematching can be done as prior by running `cgff` as in the LJ recovery and 1-site methanol tutorials.

9.4.4 Preparing CG Interaction Tables for LAMMPS simulations

Files:

- `f_forces.Pair_Interaction.total.Pair_MEO_MEO.dat`
[generated by `cgff` in previous section]
- `f_forces.Local_Density.total.LD_MEO_MEO.dat`
[generated by `cgff` in previous section]

- `translate_table.py` [provided in BOCS/force-matching/scripts]
- `quick_LD_prep.py` [provided]

Now that we have force-matching output, we need to convert it to an appropriate format / units for simulation in LAMMPS “real” units. (see section 3.1.3) This includes doing the unit conversion and padding the output.

For the pair interaction this can be done as follows:

```
# Step 1) Pad nb table with BOCS tables tool
tables f_forces.Pair_Interaction.total.Pair_MEO_MEO.dat table_MEO_MEO.xvg nb \
Bspline 4.00

#Step 2) Translate to LAMMPS format with BOCS provided python script
python /path/to/translate_table.py table_MEO_MEO.xvg nb lammps_nb_MEO_MEO.table \
MEOME0
```

For the local density interaction we have provided a very naive python script that pads the LD forces with a constant force before translating to LAMMPS format. The script requires the numpy library and is found inside of the prepare_tables folder in this tutorial and can be used as follows:

```
python quick_LD_prep.py f_forces.Local_Density.total.LD_MEO_MEO.dat
```

9.4.5 Running CG NPT LAMMPS Simulations with LD Potentials

Files:

- `lammps_nb_MEO_MEO.table` [generated by `translate_table.py` in previous section]
- `LD_table.MEO.MEO.dat` [generated by `quick_LD_prep.py` in previous section]
- `cg_methanol.bulk.lmp` [provided]
- `cg_methanol.bulk.LD_ff_input` [provided]
- `cg_methanol.data` [provided]

The lammps simulation can be run on 20 processors and should take about 2 hours. It should be run using mpi style parallelization from a folder that contains all of the above mentioned files.

```
mpirun -np 20 /path/to/lmp -in cg_methanol.bulk.lmp
```

Since a major benefit of using local density potentials includes reproducing the atomistic pressure volume equation of state, the input script `cg_methanol.bulk.lmp` includes a lammps fix to print out the time correlated system pressure and volume in bar/nm³. This file “PV.dat” can be compared to a “PV.xvg” file calculated using the gromacs `energy` tool. Sample results are depicted below and provided in the `sample_results` folder of this tutorial.

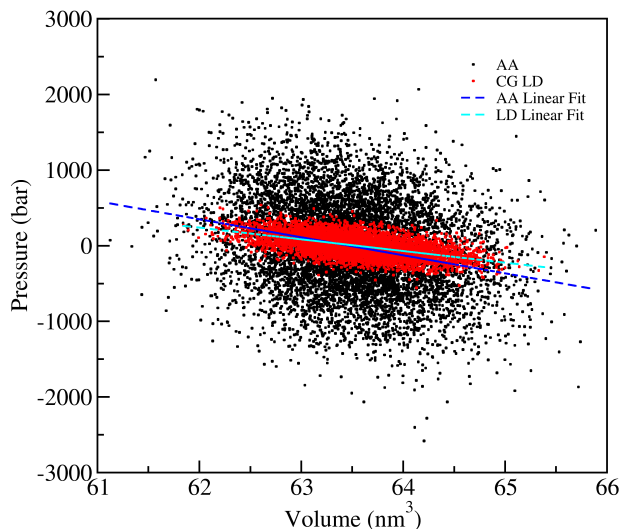


Figure 4: Atomistic and CG LD pressure volume equations of state obtained from 1 bar simulations of each. Dots represent sampled pressures at each volume, lines present a linear best fit of each scatter data (obtained from xmgrace).

9.5 External-Fields Recovery Tutorial

Directory:

- `BOCS/tutorials/Ext_Field_Tutorial`

9.5.1 Overview

In this tutorial we do no coarse-graining, instead we demonstrate how to use BOCS to construct `External_Potential` interactions using a simple system of ideal gas particles in a fixed volume under an external potential.

We use this simple system to demonstrate the following concepts:

- Working with LAMMPS data primarily
- Using structural data instead of forces (Needs a lot more data to work)
- Force-Matching External Fields

As in the LJ tutorial, the workflow is simply to run a simulation to obtain data, then use that data to infer the interactions consistent with the simulation data. However, this tutorial's workflow is primarily in LAMMPS and has two alternative ways to generate simulation data:

- Langevin Thermostat - “fix langevin” - Creates a trajectory with noisy forces saved, but obtains the correct canonical distribution for systems with fewer degrees of freedom / not a lot of thermal exchange between particles.

-We use **distribution(structure)** data from this trajectory to determine interactions.

-The input script is `in.langevin` and it generates a long trajectory.

- Shinoda’s Nose-Hoover chain - “fix nvt” - Creates a trajectory with only conservative force information saved, but generates the incorrect canonical distribution for ideal gas system.

-We use **force data** from this trajectory to determine interactions.

-The input script provided is `in.fixnvt`, it generates a much shorter trajectory. (approx 1/50th the data in the structural distribution)

In practice, BOCS simultaneously parameterizes interactions from distributional and conservative force data when both are available. So the procedure for each and the files generated are the same.

9.5.2 External-Fields Recovery Procedure

Atomistic Simulation .



Figure 5: Simulation snapshot of 3 ideal gases of different particles types under an external field in the horizontal (z) dimension.

Files:

- `in.langevin` (In `Ext_Field_Tutorial/simulation/structure_based`)
- `in.fixnvt` (In `Ext_Field_Tutorial/simulation/force_based`)

Providing detailed instructions on the use of LAMMPS is beyond the scope of this tutorial, however we provide overviews of each input script here as a “quick start”. We begin by noting that generally LAMMPS is a script based MD engine wherein each command of the input script (`in.*` provided) sets up a different aspect of the simulation. The input scripts attached set up a simulation of 1200 particles, (400 A type particles, 400 B type particles and 400 C type particles) in an anisotropic box with an elongated z dimension. Each particle

type interacts with its own external field, a simply 12-6 LJ potential along the z-dimension of the simulation box. We have provided comments within teach input script to explain how each line contributes to setting this simulation up, see [LAMMPS's user guide](#) for up to date and complete information about each command and parameters provided.

While developing this tutorial we ran each on a cluster using 4 processors for 2 minutes and 2.5 hours (respectively) with the following commands:

```
$ mpirun -np 4 lmp -in in.fixnvt
$ mpirun -np 4 lmp -in in.langevin
```

Each simulation input generates a BOCS formatted LAMMPS trajectory, `bocs_traj.lmp`. This output file is then used as the input for force-matching.

Force-Matching

Files:

1. `par.txt`
2. `1200.btp`
3. `inp.txt` <--- Must be edited to link your desired trajectory (`bocs_traj.lmp` from structure or force folder)

The syntax within `par.txt` is explained by the BOCS style `!comments` included within the tutorial files provided, more details on using `cgff` with this option are provided in section 5.3 of this manual. Directions for generating and understanding the `.btp` file provided are in section 3.2 of this manual.

We force-matched the resultant structure based and force-based trajectories for less than 1 and 40 minutes (respectively) on 120 processors using the command:

```
mpirun -np 120 cgff
```

Results for each are presented in figure 6 and 7 below, which each have an `.agr` text file that can be plotted with `xmgrace` stored in `Ext_Field_Tutorial/cgff_results`.

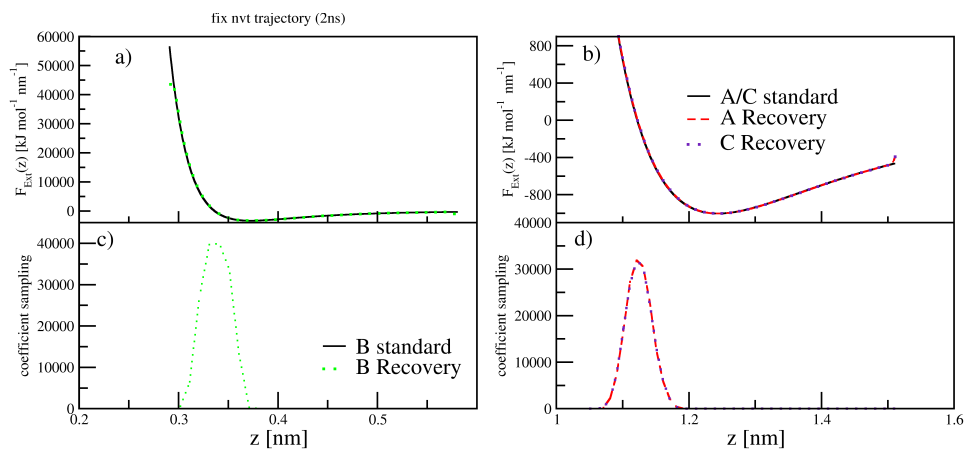


Figure 6: In this figure, force recoveries are obtained in `fixnvt`'s data, which saves conservative force information.

The top panels, a) and b) show the external field force simulated (black) and recovered (colors). The bottom panels c) and d) show the un-normalized distribution of data for each force coefficient matched. The left panels (a/c) and the right panels (b/d) separate data for different BOCS particle types.

The data for this image is available under “`Ext_Field_Tutorial/force_tutorial_results.agr`” .

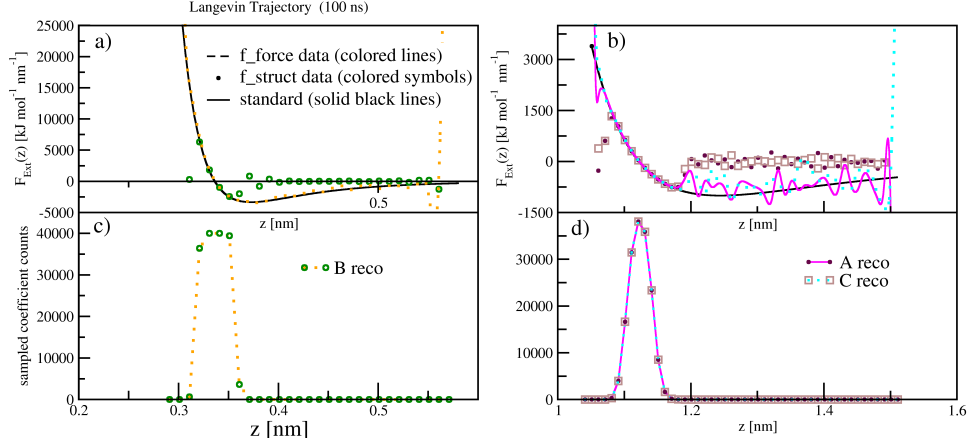


Figure 7: In this figure, force recoveries are obtained in langevin's data, where conservative force information has been obstructed by the saved random forces. Two sets of recoveries are shown, “f_force” data (lines) , which uses the noisy random forces saved, and “f_struct” data (symbols), which only uses structural distributions sampled along the z dimension of the box to determine the interaction.

The top panels, a) and b) show the external field force simulated (black) and recovered in each attempt (colors). The bottom panels c) and d) show the un-normalized distribution of data for each force coefficient matched. The left panels (a/c) and the right panels (b/d) separate data for different BOCS particle types.

Note that the structural recovery is accurate only where there is sampling, and zero outside of it. Conversely the force data, even noised, is reasonably accurate for a longer range. The data for this image is available under “Ext_Field_Tutorial/cgff.results/noisy_forces_struct_tutorial.agr” .